

Creational Patterns RoadmapJava

DP Lab Assignment (Factory, Abstract Factory, Builder).pdf

PDF

This is our full project description for the Java creational pattern lab task. Now we need to make a proper road map to complete this project. Our main goal is not to write code; rather, our goal is to learn every concept of creational design patterns and get the ability to design and review codes. We will divide this project into a small number of milestones, and then on each milestone we will first learn the concept from <https://refactoring.guru/design-patterns/creational-patterns>, this website, and then write code. We complete one milestone by one. I will give you more moving on to the next milestone once I finish one. On each milestone you need to add MCQ questions regarding the milestone; answering these questions should unlock my critical thinking on these design patterns.

Show moreShow less

I'll turn your assignment into a learning-first roadmap: milestone goals, what to read first, what to implement after, review checkpoints, and MCQs that force design thinking rather than memorization.

I found the official pattern definitions and mapped them against your assignment requirements. The assignment is clearly asking for Factory Method, Abstract Factory, Builder, plus a source strategy/inversion-of-control layer inside one online exam subsystem.

Perfect. We will follow a **learning-first roadmap**, not a code-first roadmap.

Your assignment requires an **Online Examination Subsystem** using **Factory Method**, **Abstract Factory**, **Builder**, and **Question Sourcing through dependency injection / inversion of control**. It also expects a final dashboard output, UML diagram, source-code link, AI-chat record, and the ability to explain the solution clearly.

DP Lab Assignment (Factory, Abs...

Refactoring.Guru describes creational patterns as object-creation mechanisms that improve flexibility and reuse; specifically, **Factory Method** lets subclasses decide which product to create, **Abstract Factory** creates related object families, and **Builder** constructs complex objects step by step.

 Refactoring Guru+3

Our Roadmap

Milestone 1: Understand the Problem + Pattern Map

Goal: Before writing code, understand *why* each pattern is needed.

We will identify:

Requirement	Pattern Needed	Why
Create Practice Quiz, Midterm, Final without changing main engine	Factory Method	Different exam types are products
Create MCQ/Essay/Programming question + renderer + evaluator together	Abstract Factory	Each question type is a family

Requirement	Pattern Needed	Why
Build exam settings step by step	Builder	Avoid huge constructors
Choose question source: manual, bank, faker	Dependency Injection / Strategy-like sourcing	Source can change without changing factories

Deliverable:

A simple architecture map showing major interfaces/classes, no heavy code yet.

MCQs for Milestone 1

- Why is this project not suitable for writing all object creation directly inside **Main.java**?
 - It makes the code shorter
 - It violates separation of responsibility and makes extension harder
 - Java does not allow object creation in main
 - It improves abstraction automatically
- Which requirement best represents the Open/Closed Principle?
 - We can edit **Main.java** every time we add a new exam
 - We can add a new exam type without modifying the existing core engine
 - We must close the program after execution
 - We must use private classes only
- Which part of the project is a "family of related objects"?
 - Exam, ExamFactory, Main**
 - Question, Renderer, Evaluator** for one question type
 - String, int, boolean**
 - Scanner, System.out, ArrayList**

Milestone 2: Factory Method — Exam Provisioning

Concept to learn first: Factory Method.

Main idea:

The client should not write:

Java

```
Exam exam = new MidtermExam();
```

Instead, it should depend on an abstract creator:

Java

```
ExamFactory factory = new MidtermExamFactory();
Exam exam = factory.createExam();
```

Classes we will design:

```
Exam
├── PracticeQuiz
├── MidtermExam
└── FinalExam

ExamFactory
```

```

├─ PracticeQuizFactory
├─ MidtermExamFactory
└─ FinalExamFactory

```

Learning target:

You should be able to explain the difference between:

- Product
- Concrete Product
- Creator
- Concrete Creator

Code target:

Create exam objects without directly depending on concrete exam classes inside the client.

Review checklist:

- Does `Main.java` depend on `Exam`, not directly on `MidtermExam`?
- Can we add `CertificationExam` without editing existing exam classes?
- Is there any unnecessary `if-else` or `switch` for exam creation?

MCQs for Milestone 2

1. In this project, which one is the Factory Method "Product" interface?
 - A. `Exam`
 - B. `ExamFactory`
 - C. `Main`
 - D. `ExamBuilder`
2. Which class should contain the `createExam()` method?
 - A. `QuestionRenderer`
 - B. `ExamFactory`
 - C. `ExamConfiguration`
 - D. `QuestionSource`
3. What is the biggest design benefit of Factory Method here?
 - A. It makes the console output colorful
 - B. It hides exam creation logic from the client
 - C. It removes the need for interfaces
 - D. It stores questions in a database

Milestone 3: Abstract Factory — Question Subsystem

Concept to learn first: Abstract Factory.

Main idea:

A question is not only text. Each question type has related components:

```

Question Entity
Renderer
Evaluator

```

So for MCQ, we need:

```
MCQQuestion
MCQRenderer
MCQEvaluator
MCQFactory
```

For Essay:

```
EssayQuestion
EssayRenderer
EssayEvaluator
EssayFactory
```

Classes we will design:

```
Question
QuestionRenderer
QuestionEvaluator
QuestionFactory

MCQFactory
TrueFalseFactory
EssayFactory
ProgrammingFactory
```

Learning target:

You should understand why Abstract Factory is better than creating **Question**, **Renderer**, and **Evaluator** separately in random places.

Code target:

Each concrete question factory should create a matching question family.

Example:

```
Java
QuestionFactory factory = new MCQFactory(source);

Question q = factory.createQuestion();
QuestionRenderer r = factory.createRenderer();
QuestionEvaluator e = factory.createEvaluator();
```

Review checklist:

- Does **MCQFactory** create only MCQ-related objects?
- Can we add **FillBlankFactory** later without editing old factories?
- Are renderer and evaluator matched with the correct question type?

MCQs for Milestone 3

1. Why is Abstract Factory suitable for the question subsystem?
 - A. Because all question types have unrelated objects
 - B. Because each question type has a related family: question, renderer, evaluator
 - C. Because Java requires factories for interfaces
 - D. Because it makes all classes static

2. Which group is a proper Abstract Factory family?
 - A. MCQQuestion, EssayRenderer, ProgrammingEvaluator
 - B. MCQQuestion, MCQRenderer, MCQEvaluator
 - C. Exam, ExamFactory, ExamBuilder
 - D. Main, Scanner, System.out
3. What mistake would break the Abstract Factory idea?
 - A. Creating a renderer
 - B. Passing a source object
 - C. Returning `EssayEvaluator` from `MCQFactory`
 - D. Using interfaces

Milestone 4: Builder — Exam Configuration

Concept to learn first: Builder.

Main idea:

Exam configuration has many optional values:

```
title
duration
passingScore
negativeMarking
questionShuffle
autoSubmit
calculatorAllowed
```

A constructor like this is bad:

```
Java
new ExamConfiguration("OOP Midterm", 60, 50, true, true, true, false);
```

It is hard to read and easy to make mistakes.

Instead, Builder allows:

```
Java
ExamConfiguration config = new ExamBuilder()
    .title("OOP Midterm Exam")
    .duration(60)
    .passingScore(50)
    .enableNegativeMarking()
    .enableQuestionShuffle()
    .enableAutoSubmit()
    .disableCalculator()
    .build();
```

Learning target:

You should understand constructor bloat, method chaining, immutability, and validation.

Code target:

Build an `ExamConfiguration` object step by step.

Review checklist:

- Is the configuration readable?

- Does `build()` return the final object?
- Can invalid values be prevented, such as negative duration?
- Is the final object protected from random changes?

MCQs for Milestone 4

1. Why do we use Builder for `ExamConfiguration`?
 - A. Because it has many optional/customizable fields
 - B. Because Java cannot use constructors
 - C. Because Builder replaces inheritance
 - D. Because Builder is only for database objects
2. What does method chaining mean?
 - A. Calling unrelated static methods
 - B. Calling methods one after another on the same builder object
 - C. Calling methods from different packages
 - D. Calling private methods only
3. Which constructor problem does Builder solve?
 - A. Too many interfaces
 - B. Too many parameters that are hard to read and maintain
 - C. Too many subclasses
 - D. Too many loops

Milestone 5: Question Source + Final Integration

Concept to learn first: Dependency Injection / Inversion of Control idea.

Main idea:

Question factories should not decide where questions come from. They should receive a `QuestionSource`.

```

QuestionSource
├── ManualQuestionSource
├── BankQuestionSource
└── FakerQuestionSource
  
```

Then:

```

Java
QuestionSource source = new BankQuestionSource();
QuestionFactory mcqFactory = new MCQFactory(source);
  
```

Now we can switch source mode without rewriting the question factories.

Final workflow in `Main.java`:

1. Choose exam factory.
2. Choose question source.
3. Create question factories.
4. Build exam configuration.

5. Generate dashboard output.
6. Review architecture.
7. Prepare UML and submission PDF.

Review checklist:

- Is `QuestionSource` injected into question factories?
- Can we switch from Bank mode to Faker mode easily?
- Does `Main.java` only orchestrate, not contain heavy business logic?
- Does the final console output match the assignment dashboard style?

MCQs for Milestone 5

1. Why should `MCQFactory` receive a `QuestionSource` instead of creating questions directly?
 - A. To make the source flexible and replaceable
 - B. To make the code longer
 - C. To avoid using interfaces
 - D. To prevent object creation
2. Which design is better?
 - A. `MCQFactory` always creates hardcoded MCQs internally
 - B. `MCQFactory` receives a `QuestionSource` and asks it for question data
 - C. `Main.java` creates every question, renderer, and evaluator manually
 - D. `QuestionRenderer` creates the exam
3. What is the main responsibility of `Main.java` in the final system?
 - A. Store all logic
 - B. Create every concrete object manually
 - C. Orchestrate the system workflow
 - D. Replace all factories

How We Will Work on Each Milestone

For every milestone, we will follow this sequence:

1. **Learn the pattern concept**
2. **Map the concept to this exam project**
3. **Draw the mini class structure**
4. **Write small code only after the design is clear**
5. **Review the code like an examiner**
6. **Answer MCQs before moving forward**

I will keep the MCQ answers separate so you actually think through them first.

Start with **Milestone 1**, and I'll teach the concept, architecture map, and first MCQ set properly before we write any code.

Love this approach — learning the *reasoning* behind the patterns so you can design and review, not just produce a passing artifact. That's the right instinct for someone who'll be reviewing other people's code. Here's the structure.

How each milestone works

Four beats, every time:

1. **Read** the specific concept on refactoring.guru (I'll point you to the exact page).
2. **Answer the MCQs** I give you — these are designed to be *non-obvious*, so wrong answers are where the real learning happens. Paste your letters *and* a sentence of reasoning each; the reasoning is where I can catch a shaky mental model.
3. **Write the code** for that part of the assignment.
4. **Review together** — you explain your design back to me (this also rehearses you for the "must be able to explain or zero marks" deliverable).

You drive the pace. Ping me when you've finished one and we move on.

The roadmap (5 milestones)

Milestone 1 — Foundations + Factory Method (Part A)

Why creational patterns exist at all (the OCP problem they solve), then Factory Method specifically. You build **Exam**, **ExamFactory**, and the three product/creator pairs. Outcome: you can add **CertificationExam** later without touching the engine.

Milestone 2 — Abstract Factory (Part B)

The hardest part. Families of related objects (**Question** + **Renderer** + **Evaluator**) that must stay consistent. You build the four parallel families (MCQ, True/False, Essay, Programming) and their sub-factories. Outcome: you understand *why* this isn't just "four Factory Methods stapled together."

Milestone 3 — Builder (Part C)

Constructing a complex immutable config without a 7-argument constructor. You build **ExamConfiguration** + a fluent **ExamBuilder**. Outcome: you can articulate when Builder beats Factory and beats a plain constructor.

Milestone 4 — Question Sourcing + Dependency Injection (Part D)

Inversion of Control via the **QuestionSource** interface; **BankQuestionSource** and **FakerQuestionSource** injected into your factories. Outcome: you see how DI keeps the Abstract Factory ignorant of *where* data comes from.

Milestone 5 — Integration, Orchestration & Deliverables

Wire everything in **Main.java**, produce the dashboard output, draw the UML, prep the GitHub link + explanation. Outcome: the full system + submission package.

Milestone 1 starts now

Read first: the overview at refactoring.guru/design-patterns/creational-patterns, then the full

refactoring.guru/design-patterns/factory-method page (problem, solution, structure, and the "Pros and Cons" section especially).

****Then answer these.**** Don't rush them — a couple are traps.

****Q1.**** In Factory Method, what exactly do the **subclasses** decide?

- A) The interface the product implements
- B) Which concrete product class gets instantiated
- C) How many products get created
- D) The order products are displayed

****Q2.**** You can add `CertificationExam` later without editing existing factory code. What property of Factory Method makes this possible?

- A) It uses static methods for creation
- B) The client refers to products only through an abstract type, and instantiation lives in a subclass
- C) The client holds a `switch` over exam types
- D) It caches previously created objects

****Q3.**** What truly distinguishes Factory Method from a "simple factory" (one class with `create(String type)` that switches internally)?

- A) Factory Method has no abstraction
- B) Factory Method replaces the conditional with polymorphism via subclassing
- C) The simple factory honors OCP better
- D) They're the same thing with different names

****Q4.**** In Part A, `createExam()` is declared to return `Exam`, not `MidtermExam`. Why does that return type matter?

- A) Abstract return types compile faster
- B) So the client depends on the abstraction and stays unaware of which concrete exam it received
- C) Java forces creation methods to return abstract types
- D) It enables multiple inheritance

****Q5.**** What is a **genuine cost** of using Factory Method here instead of just calling `new MidtermExam()` in the client?

- A) Runtime becomes orders of magnitude slower
- B) It spawns a parallel hierarchy of creator classes that grows alongside the product hierarchy
- C) It violates OCP
- D) It makes the products impossible to unit-test

Drop your five answers with a line of reasoning each. I'll mark them, tighten any wobbly spots, and then we write Part A together.

take this as an example. and let's start milestone one.

Show moreShow less

Great—we'll use your example structure exactly: read first, then MCQs with reasoning, then code, then review. For Milestone 1, I'll keep the scope only on **foundations + Factory Method**, matching Part A of your assignment.

Yes, we'll start **Milestone 1** now.

Milestone 1: Foundations + Factory Method

Your assignment's Part A requires an **Exam** interface, an abstract **ExamFactory**, and concrete exam/factory pairs for **PracticeQuiz**, **MidtermExam**, and **FinalExam**. The assignment also says the architecture must support OCP, meaning a new exam type like **CertificationExam** should be addable without changing the core engine.

DP Lab Assignment (Factory, Abs...

Refactoring.Guru defines creational patterns as object-creation mechanisms that improve flexibility and reuse, and Factory Method specifically gives a superclass/interface a creation method while allowing subclasses to choose which concrete object is created.  [Refactoring Guru+1](#)

Step 1: Read

Read these two pages first:

1. Creational Patterns Overview

Focus on: why object creation can become a design problem.

2. Factory Method

Focus on:

- Problem
- Solution
- Structure
- Pros and Cons
- Difference between direct **new** and factory creation

Step 2: Understand the Core Idea

Without Factory Method, your **Main.java** may look like this:

```
Java
Exam exam = new MidtermExam();
```

This works, but the client now knows the concrete class.

With Factory Method, the client says:

```
Java
ExamFactory factory = new MidtermExamFactory();
Exam exam = factory.createExam();
```

Now the client depends on the abstraction:

Java
Exam
ExamFactory

not directly on every concrete exam type.

So the mental model is:

```
Exam = Product interface
PracticeQuiz / MidtermExam / FinalExam = Concrete Products
ExamFactory = Creator
PracticeQuizFactory / MidtermExamFactory / FinalExamFactory = Concrete Creators
```

The key reasoning is this:

Factory Method does not remove object creation. It moves object creation to a controlled place, so the client does not become tightly coupled to concrete classes.

Step 3: MCQs for Milestone 1

Answer with **letter + one sentence of reasoning** for each.

Example format:

```
Q1. B – Because ...
Q2. ...
```

Q1.

In Factory Method, what exactly do the subclasses decide?

- A) The interface the product implements
- B) Which concrete product class gets instantiated
- C) How many products get created
- D) The order products are displayed

Q2.

You can add **CertificationExam** later without editing existing factory code. What property of Factory Method makes this possible?

- A) It uses static methods for creation
- B) The client refers to products only through an abstract type, and instantiation lives in a subclass
- C) The client holds a **switch** over exam types
- D) It caches previously created objects

Q3.

What truly distinguishes Factory Method from a “simple factory” — one class with `create(String type)` and a `switch` inside?

- A) Factory Method has no abstraction
- B) Factory Method replaces the conditional with polymorphism via subclassing
- C) The simple factory honors OCP better
- D) They are the same thing with different names

Q4.

In Part A, `createExam()` is declared to return `Exam`, not `MidtermExam`. Why does that return type matter?

- A) Abstract return types compile faster
- B) So the client depends on the abstraction and stays unaware of which concrete exam it received
- C) Java forces creation methods to return abstract types
- D) It enables multiple inheritance

Q5.

What is a genuine cost of using Factory Method here instead of just calling `new MidtermExam()` in the client?

- A) Runtime becomes orders of magnitude slower
- B) It creates a parallel hierarchy of creator classes that grows alongside the product hierarchy
- C) It violates OCP
- D) It makes the products impossible to unit-test

After you answer these five, I’ll mark them, explain any weak reasoning, and then we’ll write **Part A code** together.

```
// The creator class declares the factory method that must
// return an object of a product class. The creator's subclasses
// usually provide the implementation of this method.
class Dialog is
    // The creator may also provide some default implementation
    // of the factory method.
    abstract method createButton():Button

// Note that, despite its name, the creator's primary
// responsibility isn't creating products. It usually
// contains some core business logic that relies on product
// objects returned by the factory method. Subclasses can
// indirectly change that business logic by overriding the
// factory method and returning a different type of product
```

```
// from it.
method render() is
    // Call the factory method to create a product object.
    Button okButton = createButton()
    // Now use the product.
    okButton.onClick(closeDialog)
    okButton.render()

// Concrete creators override the factory method to change the
// resulting product's type.
class WindowsDialog extends Dialog is
    method createButton():Button is
        return new WindowsButton()

class WebDialog extends Dialog is
    method createButton():Button is
        return new HTMLButton()

// The product interface declares the operations that all
// concrete products must implement.
interface Button is
    method render()
    method onClick(f)

// Concrete products provide various implementations of the
// product interface.
class WindowsButton implements Button is
    method render(a, b) is
        // Render a button in Windows style.
    method onClick(f) is
        // Bind a native OS click event.

class HTMLButton implements Button is
    method render(a, b) is
        // Return an HTML representation of a button.
    method onClick(f) is
        // Bind a web browser click event.

class Application is
    field dialog: Dialog

// The application picks a creator's type depending on the
// current configuration or environment settings.
method initialize() is
    config = readApplicationConfigFile()

    if (config.OS == "Windows") then
        dialog = new WindowsDialog()
```

```
else if (config.OS == "Web") then
    dialog = new WebDialog()
else
    throw new Exception("Error! Unknown operating system.")
```

```
// The client code works with an instance of a concrete
// creator, albeit through its base interface. As long as
// the client keeps working with the creator via the base
// interface, you can pass it any creator's subclass.
method main() is
    this.initialize()
    dialog.render()
```

here we are handling if else means creating new object in Application class, should not we do it in the factory mean creator class then return to application?
Show moreShow less

Yes — excellent question. This is exactly the kind of thinking Factory Method is meant to develop.

The important point is:

Factory Method removes **if-else** from **product creation inside business logic**, but it does not magically remove every **if-else** from the whole application.

In the Refactoring.Guru example, the **Application** class is not creating the product directly:

```
Java
Button button = new WindowsButton(); // not doing this
```

Instead, it is choosing the **creator**:

```
Java
dialog = new WindowsDialog();
```

Then later the business logic works only through the abstract creator:

```
Java
dialog.render();
```

Inside **render()**, the **Dialog** class does not know whether it will get a **WindowsButton** or an **HTMLButton**:

```
Java
Button okButton = createButton();
```

That is the real Factory Method benefit.

The distinction

Bad design: client chooses product directly

```
Java
if (config.OS.equals("Windows")) {
    Button button = new WindowsButton();
} else if (config.OS.equals("Web")) {
    Button button = new HTMLButton();
}
```

Here the client knows every product class.

That means the client is tightly coupled to:

```
WindowsButton
HTMLButton
```

This is what Factory Method tries to avoid.

Better Factory Method design: client chooses creator once

```
Java
Dialog dialog;

if (config.OS.equals("Windows")) {
    dialog = new WindowsDialog();
} else if (config.OS.equals("Web")) {
    dialog = new WebDialog();
}

dialog.render();
```

Here the client only chooses the **creator**, not the product.

The product creation is delegated:

```
Java
class WindowsDialog extends Dialog {
    Button createButton() {
        return new WindowsButton();
    }
}
```

So the business logic stays inside `Dialog.render()` and works with the abstract `Button`.

Should the if-else be moved into another factory?

Sometimes, yes.

You could create a separate class like this:

```
Java
class DialogFactoryProvider {
    public static Dialog getDialog(String os) {
```

```

        if (os.equals("Windows")) {
            return new WindowsDialog();
        } else if (os.equals("Web")) {
            return new WebDialog();
        }

        throw new IllegalArgumentException("Unknown OS");
    }
}

```

Then `Application` becomes cleaner:

```

Java
dialog = DialogFactoryProvider.getDialog(config.OS);
dialog.render();

```

But now notice something important:

This `DialogFactoryProvider` is not the Factory Method pattern itself.

It is more like a **simple factory / factory provider** that chooses which concrete creator to use.

The Factory Method pattern is still here:

```

Java
abstract class Dialog {
    abstract Button createButton();
}

```

and concrete subclasses still override it:

```

Java
class WindowsDialog extends Dialog {
    Button createButton() {
        return new WindowsButton();
    }
}

```

For our exam project

In our assignment, this is acceptable:

```

Java
ExamFactory examFactory = new MidtermExamFactory();
Exam exam = examFactory.createExam();

```

Later, if we want to select exam type dynamically:

```

Java
ExamFactory examFactory;

if (examType.equals("midterm")) {
    examFactory = new MidtermExamFactory();
} else if (examType.equals("final")) {
    examFactory = new FinalExamFactory();
} else {
    examFactory = new PracticeQuizFactory();
}

```

```
Exam exam = examFactory.createExam();
```

This is okay for a small lab.

But if we want cleaner design, we can move that selection into a provider:

```
Java
class ExamFactoryProvider {
    public static ExamFactory getFactory(String examType) {
        if (examType.equalsIgnoreCase("midterm")) {
            return new MidtermExamFactory();
        } else if (examType.equalsIgnoreCase("final")) {
            return new FinalExamFactory();
        } else if (examType.equalsIgnoreCase("practice")) {
            return new PracticeQuizFactory();
        }

        throw new IllegalArgumentException("Unknown exam type");
    }
}
```

Then `Main.java` becomes:

```
Java
ExamFactory factory = ExamFactoryProvider.getFactory("midterm");
Exam exam = factory.createExam();
```

But for Milestone 1, I suggest we **do not add** `ExamFactoryProvider` **yet**. It may confuse Factory Method with Simple Factory.

For learning, we should keep the pattern pure first:

```
ExamFactory
├── PracticeQuizFactory
├── MidtermExamFactory
└── FinalExamFactory
```

Each factory creates one exam type.

Key exam explanation

You can explain it like this:

The **if-else** in **Application** chooses which creator object to use based on configuration. But the actual product creation is still delegated to the Factory Method inside the creator subclass. Factory Method does not remove all conditionals from the program; it prevents business logic from depending directly on concrete product classes.

That is the correct mental model.

After you answer these five, I'll mark them, explain any weak reasoning, and then we'll write ****Part A code**** together.

Q1: B

q2: B

q3: B
q4: B
Q5: C

You got the core idea mostly right. One answer is wrong, and it is an important one because it tests the “trade-off” mindset behind design patterns.

Good attempt. Your score: **4/5**.

MCQ Marking

Question	Your Answer	Correct	Result
Q1	B	B	Correct
Q2	B	B	Correct
Q3	B	B	Correct
Q4	B	B	Correct
Q5	C	B	Wrong

Q5 Correction

Q5 asked:

What is a genuine cost of using Factory Method here instead of just calling `new MidtermExam()` in the client?

Correct answer is:

B) It creates a parallel hierarchy of creator classes that grows alongside the product hierarchy.

Because for every product class:

```
PracticeQuiz
MidtermExam
FinalExam
```

we also create a matching creator/factory class:

```
PracticeQuizFactory
MidtermExamFactory
FinalExamFactory
```

So the cost is **more classes and more structure**.

Your answer **C: It violates OCP** is not correct because Factory Method actually helps support OCP. The assignment clearly says the system should allow new exam formats without modifying the core engine,

and Part A specifically asks for Factory Method to decouple the creation of examination types.

DP Lab Assignment (Factory, Abs...

Milestone 1 Code: Factory Method for Exam Provisioning

For this milestone, we will implement only **Part A**.

Project Structure

Create these files:

```
src/
├── Exam.java
├── PracticeQuiz.java
├── MidtermExam.java
├── FinalExam.java
├── ExamFactory.java
├── PracticeQuizFactory.java
├── MidtermExamFactory.java
├── FinalExamFactory.java
└── Main.java
```

1. Exam.java

```
Java
public interface Exam {
    String getType();

    void displayInfo();
}
```

Explanation

`Exam` is the **Product interface**.

Every exam type must follow this contract:

```
Java
String getType();
void displayInfo();
```

So `Main.java` can work with any exam object without knowing whether it is a Practice Quiz, Midterm, or Final.

2. PracticeQuiz.java

```
Java
public class PracticeQuiz implements Exam {
    @Override
```

```
public String getType() {
    return "Practice Quiz";
}

@Override
public void displayInfo() {
    System.out.println("Type: " + getType());
    System.out.println("Description: Unlimited attempts, low stakes, short duration.");
}
}
```

3. MidtermExam.java

```
Java
public class MidtermExam implements Exam {

    @Override
    public String getType() {
        return "Midterm Exam";
    }

    @Override
    public void displayInfo() {
        System.out.println("Type: " + getType());
        System.out.println("Description: Single attempt, strict time limit, comprehensive configuration.");
    }
}
```

4. FinalExam.java

```
Java
public class FinalExam implements Exam {

    @Override
    public String getType() {
        return "Final Exam";
    }

    @Override
    public void displayInfo() {
        System.out.println("Type: " + getType());
        System.out.println("Description: High security, high stakes, definitive final evaluation.");
    }
}
```

Explanation

These three classes are **Concrete Products**.

They all implement the same `Exam` interface, but each has its own behavior.

5. ExamFactory.java

```
Java
public abstract class ExamFactory {
```

```
// This is the Factory Method.
// Subclasses decide which concrete Exam object to create.
public abstract Exam createExam();

// Optional common logic that uses the product created by the factory method.
public void showCreatedExam() {
    Exam exam = createExam();

    System.out.println("=====");
    System.out.println("EXAM CREATED SUCCESSFULLY");
    System.out.println("=====");
    exam.displayInfo();
    System.out.println("=====");
}
}
```

Explanation

ExamFactory is the **Creator**.

The most important method is:

```
Java
public abstract Exam createExam();
```

Notice the return type is **Exam**, not **MidtermExam**, not **FinalExam**.

That means the client can write:

```
Java
Exam exam = factory.createExam();
```

without knowing exactly which concrete exam class was created.

The `showCreatedExam()` method is common logic. It does not care which exam type it receives. It only knows that it receives some object that follows the **Exam** contract.

6. PracticeQuizFactory.java

```
Java
public class PracticeQuizFactory extends ExamFactory {

    @Override
    public Exam createExam() {
        return new PracticeQuiz();
    }
}
```

7. MidtermExamFactory.java

```
Java
public class MidtermExamFactory extends ExamFactory {

    @Override
    public Exam createExam() {
        return new MidtermExam();
    }
}
```

```
}
}
```

8. FinalExamFactory.java

```
Java
public class FinalExamFactory extends ExamFactory {

    @Override
    public Exam createExam() {
        return new FinalExam();
    }
}
```

Explanation

These are **Concrete Creators**.

Each creator decides which concrete product to instantiate:

```
PracticeQuizFactory → PracticeQuiz
MidtermExamFactory → MidtermExam
FinalExamFactory   → FinalExam
```

This is the heart of Factory Method.

9. Main.java

```
Java
public class Main {
    public static void main(String[] args) {

        // The client chooses a factory.
        // Later, this choice may come from user input, config file, or database.
        ExamFactory factory = new MidtermExamFactory();

        // The client does not directly create MidtermExam.
        // It asks the factory to create the exam.
        Exam exam = factory.createExam();

        System.out.println("Created Exam Type: " + exam.getType());
        exam.displayInfo();

        System.out.println();

        // Demonstrating common creator logic.
        factory.showCreatedExam();
    }
}
```

Expected Output

```
Created Exam Type: Midterm Exam
Type: Midterm Exam
Description: Single attempt, strict time limit, comprehensive configuration.

=====
EXAM CREATED SUCCESSFULLY
```

```
=====
Type: Midterm Exam
Description: Single attempt, strict time limit, comprehensive configuration.
=====
```

Why This Is Factory Method

The pattern roles are:

Pattern Role	Our Project Class
Product	Exam
Concrete Product	PracticeQuiz, MidtermExam, FinalExam
Creator	ExamFactory
Concrete Creator	PracticeQuizFactory, MidtermExamFactory, FinalExamFactory
Factory Method	createExam()

The client does not write:

```
Java
MidtermExam exam = new MidtermExam();
```

Instead, it writes:

```
Java
ExamFactory factory = new MidtermExamFactory();
Exam exam = factory.createExam();
```

This keeps the client dependent on abstraction.

Bad Code Example

This works, but it is not a good design for this assignment:

```
Java
public class Main {
    public static void main(String[] args) {

        String examType = "midterm";
        Exam exam;

        if (examType.equalsIgnoreCase("practice")) {
            exam = new PracticeQuiz();
        } else if (examType.equalsIgnoreCase("midterm")) {
            exam = new MidtermExam();
        } else if (examType.equalsIgnoreCase("final")) {
            exam = new FinalExam();
        } else {
            throw new IllegalArgumentException("Unknown exam type");
        }

        exam.displayInfo();
    }
}
```

Why This Is Weaker

The problem is not that **if-else** is always evil.

The problem is that **Main.java** now knows every concrete exam class:

```
PracticeQuiz
MidtermExam
FinalExam
```

Later, if we add:

```
CertificationExam
```

we must come back and edit this conditional logic.

Factory Method improves the design by moving concrete object creation into creator subclasses.

How to Explain This in Viva

You can say:

In Milestone 1, I used the Factory Method pattern to decouple exam creation from the client. **Exam** is the product interface, and **PracticeQuiz**, **MidtermExam**, and **FinalExam** are concrete products. **ExamFactory** declares the factory method **createExam()**, while each concrete factory overrides it to return a specific exam type. This allows the client to work with the abstract **ExamFactory** and **Exam** types instead of directly depending on concrete exam classes.

A stronger explanation:

The benefit is extensibility. If I later add **CertificationExam**, I can create **CertificationExam** and **CertificationExamFactory** without modifying the existing exam classes. The cost is that I need an extra creator hierarchy, so the number of classes increases.

Your Review Task Before Milestone 2

Explain these three things in your own words:

1. Which classes are **products** and which classes are **creators**?
2. Why does **createExam()** return **Exam** instead of **MidtermExam**?
3. What is the cost of using Factory Method here?

Once you answer those, we will move to **Milestone 2: Abstract Factory**.

1. class name ends with exam and class name ends with factory(basically i understood)
2. to make client unaware of how we creating and decouple the logics.
3. i guess the cost is we need to create lots of classes.

Nice. Your understanding is correct. I'll tighten the wording so it becomes viva-ready.

Review of Your Answers

1. Products vs Creators

Your idea:

class name ends with exam and class name ends with factory

Correct, but say it more technically:

```
Products:
PracticeQuiz
MidtermExam
FinalExam

Creator:
ExamFactory

Concrete Creators:
PracticeQuizFactory
MidtermExamFactory
FinalExamFactory
```

Viva version:

The classes that implement **Exam** are products. The classes that extend **ExamFactory** are creators because they are responsible for creating the products.

2. Why does `createExam()` return **Exam**?

Your answer is good:

to make client unaware and decouple the logic

Better version:

`createExam()` returns **Exam** so the client depends on the abstraction, not the concrete class. The client does not need to know whether it received a **PracticeQuiz**, **MidtermExam**, or **FinalExam**.

This is the main Factory Method idea: creation is hidden behind an abstract product type.

3. Cost of Factory Method

Your answer is correct:

we need to create lots of classes

Better version:

The cost is extra structure. For every new exam product, we may also need a matching factory class, so the codebase gets more classes and a parallel creator hierarchy.

That is the trade-off. Factory Method improves extensibility, but it increases class count.

Milestone 1 Complete

You understand:

- Product vs Creator
- Why return type should be abstract
- Why Factory Method helps OCP
- The cost of using the pattern

Now we can move to **Milestone 2: Abstract Factory**.

Milestone 2: Abstract Factory

This will be harder than Factory Method.

Refactoring.Guru describes Abstract Factory as a creational pattern that creates **families of related objects** without specifying their concrete classes. It also highlights that the products from the same factory are compatible with each other and that the client avoids tight coupling with concrete products.

 refactoring.guru+1

In our assignment, the family is:

Question + Renderer + Evaluator

For example, MCQ is not just one class.

It has:

MCQQuestion
MCQRenderer
MCQEvaluator
MCQFactory

Essay has:

EssayQuestion
EssayRenderer
EssayEvaluator
EssayFactory

So the key idea is:

Abstract Factory does not create one object. It creates a group of related objects that are designed to work together.

Read First

Read:

refactoring.guru/design-patterns/abstract-factory

Focus on:

- Problem
- Solution
- Structure
- Product families
- Product variants
- Pros and Cons

Milestone 2 MCQs

Answer with **letter + one sentence of reasoning**.

Q1.

In our exam project, which option best represents a product family?

- A) Exam, ExamFactory, Main
- B) MCQQuestion, MCQRenderer, MCQEvaluator
- C) PracticeQuiz, MidtermExam, FinalExam
- D) ExamBuilder, ExamConfiguration, Main

Q2.

Why is Abstract Factory better than creating MCQQuestion, EssayRenderer, and ProgrammingEvaluator randomly in Main.java?

- A) It guarantees that related objects from the same family stay consistent
- B) It removes the need for interfaces
- C) It makes all objects static
- D) It prevents object creation completely

Q3.

What would be a design mistake inside MCQFactory?

- A) Returning MCQQuestion from createQuestion()
- B) Returning MCQRenderer from createRenderer()

- C) Returning **EssayEvaluator** from **createEvaluator()**
 - D) Returning objects through interfaces
-

Q4.

What is the biggest difference between Factory Method and Abstract Factory?

- A) Factory Method creates one product through subclassing; Abstract Factory creates families of related products
 - B) Factory Method is behavioral, Abstract Factory is structural
 - C) Abstract Factory never uses interfaces
 - D) Factory Method is only used in GUI apps
-

Q5.

Suppose we add a new question type called **FillBlank**. In Abstract Factory style, what should we add?

- A) Only **FillBlankQuestion**
 - B) **FillBlankQuestion**, **FillBlankRenderer**, **FillBlankEvaluator**, and **FillBlankFactory**
 - C) Only one method inside **Main.java**
 - D) A new **ExamFactory**
-

Q6.

Why does **QuestionFactory** return interfaces like **QuestionRenderer** and **QuestionEvaluator** instead of concrete classes?

- A) So the client can work with abstractions and remain unaware of concrete family classes
 - B) Because Java does not allow concrete return types
 - C) Because interfaces are faster at runtime
 - D) Because evaluators cannot be instantiated
-

Q7.

Which statement is most accurate?

- A) Abstract Factory is useful when objects are unrelated
 - B) Abstract Factory is useful when the client must create compatible objects from the same family
 - C) Abstract Factory is only useful when there is exactly one product
 - D) Abstract Factory is the same as a constructor
-

Send your answers like this:

Q1. B -- Because ...
 Q2. A -- Because ...
 ...

After that, I'll mark them and we'll write **Part B code** together.

Q1: B -- because we need to group by similar kind of product.
 Q2: A -- because it helps to group similar products and always return this type so that type no one mistakenly receives midterm renderer in mcq exam.
 Q3: C—because it must return mcq product
 Q4: A
 Q5: B -- because it just introduce a new family.
 Q6: A
 Q7: B
 Show moreShow less

Your answers show the right mental model: family consistency is the main point of Abstract Factory. I'll now mark them, then we'll write Part B code in a way that can be refactored smoothly when we add **QuestionSource** in Milestone 4.

Excellent. Your Milestone 2 MCQ score: **7/7** 

You understood the main idea: **Abstract Factory protects family consistency**. Refactoring.Guru defines Abstract Factory as creating families of related objects without specifying their concrete classes, and that is exactly what our question subsystem needs.  refactoring.guru

Your assignment says each question is not just text; it has three responsibilities: **Question Entity**, **Renderer Entity**, and **Evaluator Entity**. It also requires MCQ, True/False, Essay, and Programming question families.

DP Lab Assignment (Factory, Abs...

Milestone 2 Code: Abstract Factory for Question Subsystem

For this milestone, we will build:

```
Question
QuestionRenderer
QuestionEvaluator
QuestionFactory

MCQQuestion
MCQRenderer
MCQEvaluator
MCQFactory

TrueFalseQuestion
TrueFalseRenderer
TrueFalseEvaluator
TrueFalseFactory

EssayQuestion
EssayRenderer
EssayEvaluator
EssayFactory

ProgrammingQuestion
```

```
ProgrammingRenderer
ProgrammingEvaluator
ProgrammingFactory
```

For now, questions will be hardcoded inside factories. In **Milestone 4**, we will refactor this so factories receive `QuestionSource` through dependency injection.

1. Question.java

```
Java
public interface Question {
    String getType();

    String getText();

    int getPoints();

    String getDifficulty();
}
```

Why extra methods?

The assignment only shows `getType()`, but the final dashboard needs:

```
Type
Points
Difficulty
```

So we add these methods early to make the final integration easier.

2. QuestionRenderer.java

```
Java
public interface QuestionRenderer {
    void render(Question question);
}
```

3. QuestionEvaluator.java

```
Java
public interface QuestionEvaluator {
    int evaluate(Question question, String answer);
}
```

4. QuestionFactory.java

```
Java
public interface QuestionFactory {
    Question createQuestion();
}
```

```

    QuestionRenderer createRenderer();

    QuestionEvaluator createEvaluator();
}

```

This is the **Abstract Factory**.

It creates a full family:

Question + Renderer + Evaluator

MCQ Family

5. MCQQuestion.java

Java

```

public class MCQQuestion implements Question {
    private final String text;
    private final String[] options;
    private final String correctOption;
    private final int points;
    private final String difficulty;

    public MCQQuestion(String text, String[] options, String correctOption, int points, String
difficulty) {
        this.text = text;
        this.options = options;
        this.correctOption = correctOption;
        this.points = points;
        this.difficulty = difficulty;
    }

    @Override
    public String getType() {
        return "MCQ";
    }

    @Override
    public String getText() {
        return text;
    }

    public String[] getOptions() {
        return options;
    }

    public String getCorrectOption() {
        return correctOption;
    }

    @Override
    public int getPoints() {
        return points;
    }

    @Override
    public String getDifficulty() {
        return difficulty;
    }
}

```

6. MCQRenderer.java

Java

```
public class MCQRenderer implements QuestionRenderer {

    @Override
    public void render(Question question) {
        if (!(question instanceof MCQQuestion)) {
            throw new IllegalArgumentException("MCQRenderer can only render MCQQuestion.");
        }

        MCQQuestion mcq = (MCQQuestion) question;

        System.out.println("[MCQ Question]");
        System.out.println(mcq.getText());

        String[] options = mcq.getOptions();

        for (int i = 0; i < options.length; i++) {
            char optionLetter = (char) ('A' + i);
            System.out.println(optionLetter + " " + options[i]);
        }
    }
}
```

7. MCQEvaluator.java

Java

```
public class MCQEvaluator implements QuestionEvaluator {

    @Override
    public int evaluate(Question question, String answer) {
        if (!(question instanceof MCQQuestion)) {
            throw new IllegalArgumentException("MCQEvaluator can only evaluate MCQQuestion.");
        }

        MCQQuestion mcq = (MCQQuestion) question;

        if (mcq.getCorrectOption().equalsIgnoreCase(answer.trim())) {
            return mcq.getPoints();
        }

        return 0;
    }
}
```

8. MCQFactory.java

Java

```
public class MCQFactory implements QuestionFactory {

    @Override
    public Question createQuestion() {
        return new MCQQuestion(
            "Which design pattern creates families of related objects?",
            new String[]{"Factory Method", "Abstract Factory", "Builder", "Singleton"},
            "B",
            2,
            "Medium"
        );
    }

    @Override
    public QuestionRenderer createRenderer() {
        return new MCQRenderer();
    }
}
```

```

@Override
public QuestionEvaluator createEvaluator() {
    return new MCQEvaluator();
}
}

```

True/False Family

9. TrueFalseQuestion.java

```

Java
public class TrueFalseQuestion implements Question {
    private final String text;
    private final boolean correctAnswer;
    private final int points;
    private final String difficulty;

    public TrueFalseQuestion(String text, boolean correctAnswer, int points, String difficulty) {
        this.text = text;
        this.correctAnswer = correctAnswer;
        this.points = points;
        this.difficulty = difficulty;
    }

    @Override
    public String getType() {
        return "True/False";
    }

    @Override
    public String getText() {
        return text;
    }

    public boolean getCorrectAnswer() {
        return correctAnswer;
    }

    @Override
    public int getPoints() {
        return points;
    }

    @Override
    public String getDifficulty() {
        return difficulty;
    }
}

```

10. TrueFalseRenderer.java

```

Java
public class TrueFalseRenderer implements QuestionRenderer {

    @Override
    public void render(Question question) {
        if (!(question instanceof TrueFalseQuestion)) {
            throw new IllegalArgumentException("TrueFalseRenderer can only render TrueFalseQuestion.");
        }

        TrueFalseQuestion tf = (TrueFalseQuestion) question;
    }
}

```

```

        System.out.println("[True/False Question]");
        System.out.println(tf.getText());
        System.out.println("Answer: True or False");
    }
}

```

11. TrueFalseEvaluator.java

Java

```

public class TrueFalseEvaluator implements QuestionEvaluator {

    @Override
    public int evaluate(Question question, String answer) {
        if (!(question instanceof TrueFalseQuestion)) {
            throw new IllegalArgumentException("TrueFalseEvaluator can only evaluate TrueFalseQuestion.");
        }

        TrueFalseQuestion tf = (TrueFalseQuestion) question;

        String cleanedAnswer = answer.trim().toLowerCase();

        boolean userAnswer;

        if (cleanedAnswer.equals("true") || cleanedAnswer.equals("t")) {
            userAnswer = true;
        } else if (cleanedAnswer.equals("false") || cleanedAnswer.equals("f")) {
            userAnswer = false;
        } else {
            return 0;
        }

        if (userAnswer == tf.getCorrectAnswer()) {
            return tf.getPoints();
        }

        return 0;
    }
}

```

12. TrueFalseFactory.java

Java

```

public class TrueFalseFactory implements QuestionFactory {

    @Override
    public Question createQuestion() {
        return new TrueFalseQuestion(
            "Factory Method and Abstract Factory are both creational design patterns.",
            true,
            1,
            "Easy"
        );
    }

    @Override
    public QuestionRenderer createRenderer() {
        return new TrueFalseRenderer();
    }

    @Override
    public QuestionEvaluator createEvaluator() {
        return new TrueFalseEvaluator();
    }
}

```

Essay Family

13. EssayQuestion.java

Java

```
public class EssayQuestion implements Question {
    private final String text;
    private final String[] expectedKeywords;
    private final int points;
    private final String difficulty;

    public EssayQuestion(String text, String[] expectedKeywords, int points, String difficulty) {
        this.text = text;
        this.expectedKeywords = expectedKeywords;
        this.points = points;
        this.difficulty = difficulty;
    }

    @Override
    public String getType() {
        return "Essay";
    }

    @Override
    public String getText() {
        return text;
    }

    public String[] getExpectedKeywords() {
        return expectedKeywords;
    }

    @Override
    public int getPoints() {
        return points;
    }

    @Override
    public String getDifficulty() {
        return difficulty;
    }
}
```

14. EssayRenderer.java

Java

```
public class EssayRenderer implements QuestionRenderer {

    @Override
    public void render(Question question) {
        if (!(question instanceof EssayQuestion)) {
            throw new IllegalArgumentException("EssayRenderer can only render EssayQuestion.");
        }

        EssayQuestion essay = (EssayQuestion) question;

        System.out.println("[Essay Question]");
        System.out.println(essay.getText());
        System.out.println("Write a descriptive answer.");
    }
}
```

15. EssayEvaluator.java

Java

```
public class EssayEvaluator implements QuestionEvaluator {

    @Override
    public int evaluate(Question question, String answer) {
        if (!(question instanceof EssayQuestion)) {
            throw new IllegalArgumentException("EssayEvaluator can only evaluate EssayQuestion.");
        }

        EssayQuestion essay = (EssayQuestion) question;

        String cleanedAnswer = answer.toLowerCase();
        int matchedKeywords = 0;

        for (String keyword : essay.getExpectedKeywords()) {
            if (cleanedAnswer.contains(keyword.toLowerCase())) {
                matchedKeywords++;
            }
        }

        if (matchedKeywords == essay.getExpectedKeywords().length) {
            return essay.getPoints();
        }

        if (matchedKeywords > 0) {
            return essay.getPoints() / 2;
        }

        return 0;
    }
}
```

16. EssayFactory.java

Java

```
public class EssayFactory implements QuestionFactory {

    @Override
    public Question createQuestion() {
        return new EssayQuestion(
            "Explain why Abstract Factory is useful in an online exam system.",
            new String[]{"family", "renderer", "evaluator"},
            10,
            "Hard"
        );
    }

    @Override
    public QuestionRenderer createRenderer() {
        return new EssayRenderer();
    }

    @Override
    public QuestionEvaluator createEvaluator() {
        return new EssayEvaluator();
    }
}
```

Programming Family

17. ProgrammingQuestion.java

Java

```
public class ProgrammingQuestion implements Question {
    private final String text;
    private final String expectedKeyword;
    private final int points;
    private final String difficulty;

    public ProgrammingQuestion(String text, String expectedKeyword, int points, String difficulty)
    {
        this.text = text;
        this.expectedKeyword = expectedKeyword;
        this.points = points;
        this.difficulty = difficulty;
    }

    @Override
    public String getType() {
        return "Programming";
    }

    @Override
    public String getText() {
        return text;
    }

    public String getExpectedKeyword() {
        return expectedKeyword;
    }

    @Override
    public int getPoints() {
        return points;
    }

    @Override
    public String getDifficulty() {
        return difficulty;
    }
}
```

18. ProgrammingRenderer.java

Java

```
public class ProgrammingRenderer implements QuestionRenderer {

    @Override
    public void render(Question question) {
        if (!(question instanceof ProgrammingQuestion)) {
            throw new IllegalArgumentException("ProgrammingRenderer can only render
ProgrammingQuestion.");
        }

        ProgrammingQuestion programming = (ProgrammingQuestion) question;

        System.out.println("[Programming Question]");
        System.out.println(programming.getText());
        System.out.println("Write your code below.");
    }
}
```

19. ProgrammingEvaluator.java

Java

```
public class ProgrammingEvaluator implements QuestionEvaluator {

    @Override
    public int evaluate(Question question, String answer) {
        if (!(question instanceof ProgrammingQuestion)) {
            throw new IllegalArgumentException("ProgrammingEvaluator can only evaluate
ProgrammingQuestion.");
        }

        ProgrammingQuestion programming = (ProgrammingQuestion) question;

        if (answer.toLowerCase().contains(programming.getExpectedKeyword().toLowerCase())) {
            return programming.getPoints();
        }

        return 0;
    }
}
```

20. ProgrammingFactory.java

```
Java
public class ProgrammingFactory implements QuestionFactory {

    @Override
    public Question createQuestion() {
        return new ProgrammingQuestion(
            "Write a Java method that returns the larger of two numbers.",
            "Math.max",
            20,
            "Hard"
        );
    }

    @Override
    public QuestionRenderer createRenderer() {
        return new ProgrammingRenderer();
    }

    @Override
    public QuestionEvaluator createEvaluator() {
        return new ProgrammingEvaluator();
    }
}
```

21. Temporary Main.java for Milestone 2 Testing

This `Main.java` is only for testing Milestone 2. Later in Milestone 5, we will create the final dashboard-style `Main.java`.

```
Java
public class Main {
    public static void main(String[] args) {

        QuestionFactory mcqFactory = new MCQFactory();
        QuestionFactory trueFalseFactory = new TrueFalseFactory();
        QuestionFactory essayFactory = new EssayFactory();
        QuestionFactory programmingFactory = new ProgrammingFactory();

        testQuestionFamily(mcqFactory, "B");
        testQuestionFamily(trueFalseFactory, "true");
        testQuestionFamily(essayFactory, "A family contains question, renderer, and evaluator.");
        testQuestionFamily(programmingFactory, "return Math.max(a, b);");
    }
}
```

```
private static void testQuestionFamily(QuestionFactory factory, String sampleAnswer) {
    Question question = factory.createQuestion();
    QuestionRenderer renderer = factory.createRenderer();
    QuestionEvaluator evaluator = factory.createEvaluator();

    System.out.println("=====");
    renderer.render(question);

    int score = evaluator.evaluate(question, sampleAnswer);

    System.out.println("Type: " + question.getType());
    System.out.println("Points: " + question.getPoints());
    System.out.println("Difficulty: " + question.getDifficulty());
    System.out.println("Sample Score: " + score);
    System.out.println("=====");
    System.out.println();
}
}
```

Expected Output

Your output should look similar to this:

```
=====
[MCQ Question]
Which design pattern creates families of related objects?
A) Factory Method
B) Abstract Factory
C) Builder
D) Singleton
Type: MCQ
Points: 2
Difficulty: Medium
Sample Score: 2
=====

=====
[True/False Question]
Factory Method and Abstract Factory are both creational design patterns.
Answer: True or False
Type: True/False
Points: 1
Difficulty: Easy
Sample Score: 1
=====
```

The full output will continue for Essay and Programming.

Why This Is Abstract Factory

The assignment wants this structure:

```
Java
interface QuestionFactory {
    Question createQuestion();
    QuestionRenderer createRenderer();
    QuestionEvaluator createEvaluator();
}
```

That means one factory creates several related objects.

DP Lab Assignment (Factory, Abs...

Example:

Java

```
QuestionFactory factory = new MCQFactory();

Question question = factory.createQuestion();
QuestionRenderer renderer = factory.createRenderer();
QuestionEvaluator evaluator = factory.createEvaluator();
```

Here, MCQFactory gives us:

```
MCQQuestion
MCQRenderer
MCQEvaluator
```

So the family stays consistent.

Bad Code Example

This is the type of code we are avoiding:

Java

```
Question question = new MCQQuestion(
    "Which pattern creates families?",
    new String[]{"Factory Method", "Abstract Factory"},
    "B",
    2,
    "Medium"
);

QuestionRenderer renderer = new EssayRenderer();
QuestionEvaluator evaluator = new ProgrammingEvaluator();
```

This is bad because the family is broken:

```
MCQQuestion
EssayRenderer
ProgrammingEvaluator
```

The question, renderer, and evaluator do not belong together.

Abstract Factory prevents this by giving all three objects from one family factory.

Viva Explanation

You can explain Milestone 2 like this:

In this milestone, I used Abstract Factory for the question subsystem. A question is not just one object; it has a question entity, renderer, and evaluator. So I created a **QuestionFactory** interface with `createQuestion()`, `createRenderer()`, and `createEvaluator()`. Each concrete factory, such as **MCQFactory** or **EssayFactory**, returns a complete family of compatible objects. This keeps the client from accidentally mixing an MCQ question with an Essay renderer or Programming evaluator.

Stronger version:

Factory Method creates one product, but Abstract Factory creates a family of related products. In this project, the related family is **Question + Renderer + Evaluator**. That is why Abstract Factory is more suitable for Part B than using separate Factory Methods randomly.

Review Task Before Milestone 3

Answer these in your own words:

1. Why is **MCQQuestion + MCQRenderer + MCQEvaluator** called a family?
2. What mistake does Abstract Factory prevent in this milestone?
3. Why is Abstract Factory not the same as Factory Method?

After that, we'll move to **Milestone 3: Builder Pattern**.

1. because they belong to to same MCQ type questions
2. we can not mistakenly mix mcq questions with essay evaluator.
3. abstraact gives the famlies of releted type and grows both horizontally and vertically where factory grows horizontally and with subclass only.

Good. Milestone 2 is complete 

Your answers are correct, and your third answer is actually advanced — just needs cleaner wording.

Review of Your Answers

1. Why is **MCQQuestion + MCQRenderer + MCQEvaluator** a family?

Your answer:

because they belong to same MCQ type questions

Correct.

Better viva version:

They are called a family because all three objects belong to the same question variant: MCQ. **MCQQuestion** stores MCQ data, **MCQRenderer** displays MCQ format, and **MCQEvaluator** checks MCQ-style answers.

2. What mistake does Abstract Factory prevent?

Your answer:

we cannot mistakenly mix MCQ questions with Essay evaluator

Correct.

Better viva version:

Abstract Factory prevents incompatible object mixing. For example, it prevents the client from creating an **MCQQuestion** but accidentally using an **EssayRenderer** or **ProgrammingEvaluator**.

This is exactly why the assignment asks for separate question families: question entity, renderer entity, and evaluator entity must stay related.

DP Lab Assignment (Factory, Abs...

3. Why is Abstract Factory not the same as Factory Method?

Your answer:

abstract gives the families of related type and grows both horizontally and vertically where factory grows horizontally and with subclass only

Good thinking. Let's sharpen it.

Better version:

Factory Method usually creates one product through a creator subclass. Abstract Factory creates a family of related products through one factory interface. So Factory Method answers "Which single product should I create?" while Abstract Factory answers "Which compatible group of products should I create?"

About your "horizontal and vertical" idea:

Abstract Factory:

- Vertical growth: Add a new family, such as **FillBlankFactory**.
- Horizontal growth: Add a new product role to every family, such as **QuestionHint** or **QuestionValidator**.

Factory Method:

- Mainly grows by adding new product/creator pairs, such as **CertificationExam** + **CertificationExamFactory**.

So your idea is right, but in exam/viva, use the simpler sentence first.

Milestone 3 Starts Now: Builder Pattern

Your assignment's Part C requires **ExamConfiguration** and **ExamBuilder**. The goal is to construct flexible exam settings such as title, duration, passing score, negative marking, shuffle, auto-submit, and calculator allowed.

DP Lab Assignment (Factory, Abs...

Refactoring.Guru says Builder lets you construct complex objects step by step and avoids forcing many parameters into one large constructor. It is especially useful when an object has many possible configuration options.  refactoring.guru+1

Read First

Read:

refactoring.guru/design-patterns/builder

Focus on:

- Problem
- Solution
- Structure
- Director idea
- Pros and Cons
- Difference between Builder and Abstract Factory

Why Builder Is Needed Here

Without Builder, we may write:

```
Java
ExamConfiguration config = new ExamConfiguration(
    "OOP Midterm Exam",
    60,
    50,
    true,
    true,
    true,
    false
);
```

This is dangerous because the booleans are unclear:

```
Java
true, true, true, false
```

Which one means negative marking?

Which one means calculator allowed?

Which one means auto-submit?

This is called **constructor bloat** or **telescoping constructor problem**.

Builder makes it readable:

```
Java
ExamConfiguration config = new ExamBuilder()
    .title("OOP Midterm Exam")
    .duration(60)
    .passingScore(50)
    .enableNegativeMarking()
    .enableQuestionShuffle()
    .enableAutoSubmit()
    .disableCalculator()
    .build();
```

Now the construction process explains itself.

Milestone 3 MCQs

Answer with **letter + one sentence of reasoning**.

Q1.

Why is Builder suitable for `ExamConfiguration`?

- A) Because `ExamConfiguration` has many optional and customizable settings
- B) Because Java cannot create objects using constructors
- C) Because Builder is only for creating product families
- D) Because Builder removes the need for classes

Q2.

What is the main problem with this constructor?

```
Java
new ExamConfiguration("OOP Midterm", 60, 50, true, true, false, true);
```

- A) It cannot compile in Java
- B) It is hard to understand which boolean means which setting
- C) It violates inheritance
- D) It creates multiple objects automatically

Q3.

Which version is more readable?

A)

```
Java
new ExamConfiguration("Final", 120, 60, true, true, true, false);
```

B)

```
Java
new ExamBuilder()
    .title("Final")
    .duration(120)
    .passingScore(60)
    .enableNegativeMarking()
    .enableQuestionShuffle()
    .enableAutoSubmit()
    .disableCalculator()
    .build();
```

C)

```
Java
```

```
ExamConfiguration config = null;
```

D)

Java

```
String config = "Final";
```

Q4.

What should `build()` do in a Builder?

- A) Print the object only
 - B) Return the final constructed object
 - C) Delete the builder
 - D) Create unrelated product families
-

Q5.

What is one genuine cost of Builder?

- A) It makes object creation impossible
 - B) It may increase code size because we need an extra builder class
 - C) It prevents validation
 - D) It breaks encapsulation automatically
-

Q6.

Builder is different from Abstract Factory because:

- A) Builder constructs one complex object step by step; Abstract Factory creates families of related objects
 - B) Builder creates only interfaces
 - C) Abstract Factory uses no classes
 - D) They are exactly the same pattern
-

Q7.

In our project, which object should be immutable after construction?

- A) `ExamConfiguration`
 - B) `Main`
 - C) `System.out`
 - D) `Scanner`
-

Q8.

Why is immutability useful for `ExamConfiguration`?

- A) The exam rules should not randomly change after the exam is created
- B) It makes the console output larger
- C) It removes the need for methods
- D) It allows multiple inheritance

Send your answers like this:

```
Q1. A – Because ...
Q2. B – Because ...
...
```

After I mark them, we'll write **Milestone 3 code** for `ExamConfiguration` and `ExamBuilder`.

```
1. A
2. B
3. B
4. B
5. B
6. A
7. A
8. A
Show moreShow less
```

Great. Your Milestone 3 MCQ score: **8/8** 

You selected all correct answers. One thing: next time, also add **one sentence reasoning** after each answer, because your explanation is what helps me check your mental model. But your choices are fully correct.

Milestone 3 Code: Builder Pattern

Your assignment requires `ExamConfiguration` with:

```
title
duration
passingScore
negativeMarking
questionShuffle
autoSubmit
calculatorAllowed
```

and an `ExamBuilder` to construct it step by step.

DP Lab Assignment (Factory, Abs...

1. ExamConfiguration.java

Java

```
public class ExamConfiguration {
    private final String title;
    private final int duration;
    private final int passingScore;
    private final boolean negativeMarking;
    private final boolean questionShuffle;
    private final boolean autoSubmit;
    private final boolean calculatorAllowed;

    public ExamConfiguration(
        String title,
        int duration,
        int passingScore,
        boolean negativeMarking,
        boolean questionShuffle,
        boolean autoSubmit,
        boolean calculatorAllowed
    ) {
        this.title = title;
        this.duration = duration;
        this.passingScore = passingScore;
        this.negativeMarking = negativeMarking;
        this.questionShuffle = questionShuffle;
        this.autoSubmit = autoSubmit;
        this.calculatorAllowed = calculatorAllowed;
    }

    public String getTitle() {
        return title;
    }

    public int getDuration() {
        return duration;
    }

    public int getPassingScore() {
        return passingScore;
    }

    public boolean isNegativeMarking() {
        return negativeMarking;
    }

    public boolean isQuestionShuffle() {
        return questionShuffle;
    }

    public boolean isAutoSubmit() {
        return autoSubmit;
    }

    public boolean isCalculatorAllowed() {
        return calculatorAllowed;
    }

    public void displayConfiguration() {
        System.out.println("Title: " + title);
        System.out.println("Duration: " + duration + " minutes");
        System.out.println("Passing Score: " + passingScore + " Marks");
        System.out.println("Configuration Profiles:");

        System.out.println((negativeMarking ? "✓" : "x") + " Negative Marking Enabled");
        System.out.println((questionShuffle ? "✓" : "x") + " Question Shuffle Activated");
        System.out.println((calculatorAllowed ? "✓" : "x") + " Embedded Calculator Allowed");
        System.out.println((autoSubmit ? "✓" : "x") + " Auto-Submit On Timeout");
    }
}
```

Why this class is immutable

All fields are:

```
Java  
private final
```

There are only getters, no setters.

So after the exam configuration is created, nobody can randomly change the rules.

That is important because an exam's rules should stay stable after construction.

2. ExamBuilder.java

```
Java  
public class ExamBuilder {  
    private String title;  
    private int duration;  
    private int passingScore;  
    private boolean negativeMarking;  
    private boolean questionShuffle;  
    private boolean autoSubmit;  
    private boolean calculatorAllowed;  
  
    public ExamBuilder title(String title) {  
        this.title = title;  
        return this;  
    }  
  
    public ExamBuilder duration(int duration) {  
        this.duration = duration;  
        return this;  
    }  
  
    public ExamBuilder passingScore(int passingScore) {  
        this.passingScore = passingScore;  
        return this;  
    }  
  
    public ExamBuilder enableNegativeMarking() {  
        this.negativeMarking = true;  
        return this;  
    }  
  
    public ExamBuilder disableNegativeMarking() {  
        this.negativeMarking = false;  
        return this;  
    }  
  
    public ExamBuilder enableQuestionShuffle() {  
        this.questionShuffle = true;  
        return this;  
    }  
  
    public ExamBuilder disableQuestionShuffle() {  
        this.questionShuffle = false;  
        return this;  
    }  
  
    public ExamBuilder enableAutoSubmit() {  
        this.autoSubmit = true;  
        return this;  
    }  
}
```

```

public ExamBuilder disableAutoSubmit() {
    this.autoSubmit = false;
    return this;
}

public ExamBuilder enableCalculator() {
    this.calculatorAllowed = true;
    return this;
}

public ExamBuilder disableCalculator() {
    this.calculatorAllowed = false;
    return this;
}

public ExamConfiguration build() {
    validate();

    return new ExamConfiguration(
        title,
        duration,
        passingScore,
        negativeMarking,
        questionShuffle,
        autoSubmit,
        calculatorAllowed
    );
}

private void validate() {
    if (title == null || title.trim().isEmpty()) {
        throw new IllegalArgumentException("Exam title cannot be empty.");
    }

    if (duration <= 0) {
        throw new IllegalArgumentException("Exam duration must be greater than 0.");
    }

    if (passingScore < 0) {
        throw new IllegalArgumentException("Passing score cannot be negative.");
    }
}
}

```

3. Temporary Main.java for Testing Builder

```

Java
public class Main {
    public static void main(String[] args) {

        ExamConfiguration config = new ExamBuilder()
            .title("OOP Midterm Exam")
            .duration(60)
            .passingScore(50)
            .enableNegativeMarking()
            .enableQuestionShuffle()
            .enableAutoSubmit()
            .disableCalculator()
            .build();

        System.out.println("=====");
        System.out.println("EXAM CONFIGURATION CREATED");
        System.out.println("=====");

        config.displayConfiguration();

        System.out.println("=====");
    }
}

```

Expected Output

```
=====
EXAM CONFIGURATION CREATED
=====
Title: OOP Midterm Exam
Duration: 60 minutes
Passing Score: 50 Marks
Configuration Profiles:
✓ Negative Marking Enabled
✓ Question Shuffle Activated
✗ Embedded Calculator Allowed
✓ Auto-Submit On Timeout
=====
```

Why This Is Builder Pattern

Without Builder, this is hard to read:

```
Java
new ExamConfiguration("OOP Midterm Exam", 60, 50, true, true, true, false);
```

The meaning of these values is unclear:

```
Java
true, true, true, false
```

Builder makes the object construction readable:

```
Java
ExamConfiguration config = new ExamBuilder()
    .title("OOP Midterm Exam")
    .duration(60)
    .passingScore(50)
    .enableNegativeMarking()
    .enableQuestionShuffle()
    .enableAutoSubmit()
    .disableCalculator()
    .build();
```

Now every step explains itself.

Viva Explanation

You can say:

I used the Builder Pattern for **ExamConfiguration** because it has many optional and customizable settings. If I used a long constructor, it would be hard to understand the boolean values. The builder allows me to construct the configuration step by step using readable method chaining. The final **ExamConfiguration** object is immutable, so exam rules cannot change randomly after construction.

Stronger version:

Builder is different from Factory Method and Abstract Factory. Factory Method creates one product through subclassing. Abstract Factory creates families of related products. Builder creates one complex object gradually and safely.

Review Task Before Milestone 4

Answer these in your own words:

1. Why is a 7-argument constructor dangerous here?
2. Why is method chaining useful in `ExamBuilder`?
3. Why should `ExamConfiguration` be immutable?

After that, we'll move to **Milestone 4: Question Sourcing + Dependency Injection**.

your are questions are two wasy don't want to answer them, another thing is teach me the builder class more deeply like how method chaining is working.

Fair point. I'll stop giving easy review questions for this part and make the next MCQs more design-trap based. For now, let's slow down and deeply understand **how Builder and method chaining actually work in memory**.

Absolutely. Your point is valid — those review questions were too easy. From now on, I'll make the questions more like **design review traps**, not memorization.

Now let's deeply understand the **Builder class** and especially **method chaining**.

Your assignment specifically requires the Builder Pattern for creating highly customizable, immutable exam configuration objects step by step, avoiding constructor bloat. It also says the final workflow should use chained method calls through an `ExamBuilder` pipeline.

DP Lab Assignment (Factory, Abs...

1. What problem is Builder solving?

Imagine this:

```
Java
ExamConfiguration config = new ExamConfiguration(
    "OOP Midterm Exam",
    60,
    50,
    true,
    true,
    true,
    false
);
```

This creates the object, but the problem is readability.

What do these mean?

```
Java
```

```
true, true, true, false
```

You have to remember the constructor order:

```
Java
negativeMarking,
questionShuffle,
autoSubmit,
calculatorAllowed
```

That is dangerous because this mistake is easy:

```
Java
new ExamConfiguration("OOP Midterm", 60, 50, true, true, false, true);
```

Maybe you wanted:

```
autoSubmit = true
calculatorAllowed = false
```

But accidentally you wrote:

```
autoSubmit = false
calculatorAllowed = true
```

The code still compiles, but the exam rules become wrong. That is the danger.

Builder solves this by making each step named:

```
Java
ExamConfiguration config = new ExamBuilder()
    .title("OOP Midterm Exam")
    .duration(60)
    .passingScore(50)
    .enableNegativeMarking()
    .enableQuestionShuffle()
    .enableAutoSubmit()
    .disableCalculator()
    .build();
```

Now the code reads like English.

2. The Builder is not the final object

Very important:

```
ExamBuilder ≠ ExamConfiguration
```

The builder is a temporary construction helper.

Think of it like this:

```
ExamBuilder = form/factory worker
ExamConfiguration = final exam rule sheet
```

The builder collects values first:

```
title = "OOP Midterm Exam"
duration = 60
passingScore = 50
negativeMarking = true
questionShuffle = true
autoSubmit = true
calculatorAllowed = false
```

Then `build()` creates the final object:

```
Java
return new ExamConfiguration(...);
```

After `build()`, we should use the `ExamConfiguration`, not the builder.

3. How method chaining works

Look at one builder method:

```
Java
public ExamBuilder title(String title) {
    this.title = title;
    return this;
}
```

The important part is:

```
Java
return this;
```

`this` means:

the current object that is calling the method.

So when we write:

```
Java
new ExamBuilder().title("OOP Midterm Exam")
```

Java does this:

```
Java
ExamBuilder builder = new ExamBuilder();
builder.title("OOP Midterm Exam");
```

But because `title()` returns the same builder object, we can immediately call another method on it:

```
Java
new ExamBuilder()
    .title("OOP Midterm Exam")
    .duration(60)
```

This works because:

```
Java
.title("OOP Midterm Exam")
```

returns:

```
Java
this
```

and `this` is still an `ExamBuilder`.

So Java sees it like:

```
Java
ExamBuilder builder = new ExamBuilder();

builder.title("OOP Midterm Exam");
builder.duration(60);
builder.passingScore(50);
builder.enableNegativeMarking();

ExamConfiguration config = builder.build();
```

These two versions are logically the same.

4. Step-by-step memory flow

When you write:

```
Java
ExamConfiguration config = new ExamBuilder()
    .title("OOP Midterm Exam")
    .duration(60)
    .passingScore(50)
    .enableNegativeMarking()
    .enableQuestionShuffle()
    .enableAutoSubmit()
    .disableCalculator()
    .build();
```

This is what happens internally.

Step 1

```
Java
new ExamBuilder()
```

A new builder object is created in memory.

At first, its fields have default values:

```
title = null
duration = 0
passingScore = 0
```

```
negativeMarking = false
questionShuffle = false
autoSubmit = false
calculatorAllowed = false
```

Step 2

```
Java
.title("OOP Midterm Exam")
```

This method runs:

```
Java
public ExamBuilder title(String title) {
    this.title = title;
    return this;
}
```

Now builder state becomes:

```
title = "OOP Midterm Exam"
duration = 0
passingScore = 0
negativeMarking = false
questionShuffle = false
autoSubmit = false
calculatorAllowed = false
```

Then it returns the same builder object.

Step 3

```
Java
.duration(60)
```

This method runs:

```
Java
public ExamBuilder duration(int duration) {
    this.duration = duration;
    return this;
}
```

Now builder state becomes:

```
title = "OOP Midterm Exam"
duration = 60
passingScore = 0
negativeMarking = false
questionShuffle = false
autoSubmit = false
calculatorAllowed = false
```

Again, it returns the same builder object.

Step 4

```
Java
.passingScore(50)
```

Now:

```
title = "OOP Midterm Exam"
duration = 60
passingScore = 50
negativeMarking = false
questionShuffle = false
autoSubmit = false
calculatorAllowed = false
```

Step 5

```
Java
.enableNegativeMarking()
```

This method does not need a parameter:

```
Java
public ExamBuilder enableNegativeMarking() {
    this.negativeMarking = true;
    return this;
}
```

Now:

```
negativeMarking = true
```

Step 6

```
Java
.disableCalculator()
```

This method sets:

```
Java
this.calculatorAllowed = false;
```

Actually, the default value is already `false`, but calling `disableCalculator()` makes the intention clear.

Step 7

```
Java
.build()
```

Now `build()` runs:

```
Java
public ExamConfiguration build() {
    validate();

    return new ExamConfiguration(
        title,
        duration,
        passingScore,
        negativeMarking,
        questionShuffle,
        autoSubmit,
        calculatorAllowed
    );
}
```

This creates the final immutable `ExamConfiguration`.

5. Why does each method return `ExamBuilder`?

Because we want the next method to be callable.

This works:

```
Java
public ExamBuilder title(String title) {
    this.title = title;
    return this;
}
```

Because the return type is `ExamBuilder`.

But this would not support chaining:

```
Java
public void title(String title) {
    this.title = title;
}
```

Then this code would fail:

```
Java
new ExamBuilder()
    .title("OOP Midterm")
    .duration(60);
```

Why?

Because `title()` returns `void`.

After this part:

```
Java
```

```
.title("OOP Midterm")
```

there is no object returned, so Java has nothing on which it can call:

```
Java
.duration(60)
```

So method chaining needs two things:

1. The method must return the builder type.
2. The method must return `this`.

6. What does this really mean?

Inside this method:

```
Java
public ExamBuilder duration(int duration) {
    this.duration = duration;
    return this;
}
```

There are two durations:

```
Java
private int duration;           // field
public ExamBuilder duration(int duration) // parameter
```

The parameter name and field name are the same.

So we use:

```
Java
this.duration
```

to mean:

the duration field of the current object

And plain:

```
Java
duration
```

means:

the parameter passed into the method

So:

```
Java
this.duration = duration;
```

means:

```
Set the builder object's duration field equal to the parameter value.
```

Example:

```
Java
.duration(60)
```

means:

```
this.duration = 60;
```

7. Why build() returns ExamConfiguration, not ExamBuilder

All setter-like methods return **ExamBuilder** because we are still building:

```
Java
.title(...)
.duration(...)
.passingScore(...)
```

But the final method returns the finished object:

```
Java
.build()
```

So:

```
Java
public ExamConfiguration build()
```

returns:

```
Java
ExamConfiguration
```

That is why this works:

```
Java
ExamConfiguration config = new ExamBuilder()
    .title("OOP Midterm")
    .duration(60)
    .passingScore(50)
    .build();
```

But this would not make sense:

```
Java
ExamBuilder config = new ExamBuilder()
    .title("OOP Midterm")
```

```
.duration(60)
.build();
```

Because `build()` does not return a builder. It returns the final configuration.

8. A deeper design issue: public constructor vs private constructor

Our earlier code had this:

```
Java
public ExamConfiguration(...)
```

That works, but it allows someone to bypass the builder:

```
Java
ExamConfiguration config = new ExamConfiguration(
    "Final",
    120,
    60,
    true,
    true,
    true,
    false
);
```

For a stricter Builder design, we can make the constructor package-private or private.

A cleaner advanced version is to use a **static nested Builder** inside `ExamConfiguration`.

Example:

```
Java
public class ExamConfiguration {
    private final String title;
    private final int duration;
    private final int passingScore;
    private final boolean negativeMarking;
    private final boolean questionShuffle;
    private final boolean autoSubmit;
    private final boolean calculatorAllowed;

    private ExamConfiguration(Builder builder) {
        this.title = builder.title;
        this.duration = builder.duration;
        this.passingScore = builder.passingScore;
        this.negativeMarking = builder.negativeMarking;
        this.questionShuffle = builder.questionShuffle;
        this.autoSubmit = builder.autoSubmit;
        this.calculatorAllowed = builder.calculatorAllowed;
    }

    public static class Builder {
        private String title;
        private int duration;
        private int passingScore;
        private boolean negativeMarking;
        private boolean questionShuffle;
        private boolean autoSubmit;
        private boolean calculatorAllowed;

        public Builder title(String title) {
```

```

        this.title = title;
        return this;
    }

    public Builder duration(int duration) {
        this.duration = duration;
        return this;
    }

    public Builder passingScore(int passingScore) {
        this.passingScore = passingScore;
        return this;
    }

    public Builder enableNegativeMarking() {
        this.negativeMarking = true;
        return this;
    }

    public Builder enableQuestionShuffle() {
        this.questionShuffle = true;
        return this;
    }

    public Builder enableAutoSubmit() {
        this.autoSubmit = true;
        return this;
    }

    public Builder disableCalculator() {
        this.calculatorAllowed = false;
        return this;
    }

    public Builder enableCalculator() {
        this.calculatorAllowed = true;
        return this;
    }

    public ExamConfiguration build() {
        validate();
        return new ExamConfiguration(this);
    }

    private void validate() {
        if (title == null || title.trim().isEmpty()) {
            throw new IllegalArgumentException("Title cannot be empty.");
        }

        if (duration <= 0) {
            throw new IllegalArgumentException("Duration must be greater than 0.");
        }

        if (passingScore < 0) {
            throw new IllegalArgumentException("Passing score cannot be negative.");
        }
    }
}

```

Then we create config like this:

```

Java
ExamConfiguration config = new ExamConfiguration.Builder()
    .title("OOP Midterm Exam")
    .duration(60)
    .passingScore(50)
    .enableNegativeMarking()
    .enableQuestionShuffle()
    .enableAutoSubmit()
    .disableCalculator()
    .build();

```

This version is stronger because nobody outside can directly call the long constructor.

9. Separate ExamBuilder vs nested Builder

For your assignment, both are acceptable.

Your assignment shows:

```
Java
class ExamBuilder {
    private ExamConfiguration config = new ExamConfiguration();
}
```

So a separate `ExamBuilder` class is closer to the assignment skeleton.

But from a design-review point of view:

Design	Advantage	Disadvantage
Separate <code>ExamBuilder</code>	Easy for beginners and matches assignment style	Can expose constructor unless controlled
Nested <code>ExamConfiguration.Builder</code>	Stronger encapsulation	Slightly harder to understand at first

For this lab, I suggest we keep **separate** `ExamBuilder`, because it is easier to explain. But we can still make `ExamConfiguration` immutable.

10. The most important mental model

Builder has three layers:

```
Client code:
Chooses readable construction steps.

Builder:
Stores temporary values and validates them.

Final object:
Receives values once and becomes immutable.
```

So this:

```
Java
ExamConfiguration config = new ExamBuilder()
    .title("OOP Midterm Exam")
    .duration(60)
    .passingScore(50)
    .enableNegativeMarking()
    .enableQuestionShuffle()
    .enableAutoSubmit()
    .disableCalculator()
    .build();
```

means:

```
Create a builder.
Set title.
Set duration.
Set passing score.
Turn on negative marking.
Turn on shuffle.
Turn on auto submit.
Turn off calculator.
Validate everything.
Create final ExamConfiguration.
```

Design-review trap to remember

Builder is useful here **not because object creation is polymorphic**.

That is Factory Method / Abstract Factory.

Builder is useful because object construction is:

```
complex
step-by-step
readable
configurable
validation-friendly
```

So in viva, say:

I used Builder because `ExamConfiguration` has many optional settings. The builder stores temporary values, each method returns `this` to enable chaining, and `build()` validates the values and returns the final immutable configuration object.

That sentence is strong.

i am using the nested builder.

Good choice. **Nested Builder is stronger** than separate `ExamBuilder` because it protects `ExamConfiguration` better.

For your project, use this design:

```
ExamConfiguration
└─ static nested class Builder
```

That means you **do not need a separate** `ExamBuilder.java` file.

Final Nested Builder Version

ExamConfiguration.java

```
Java
public class ExamConfiguration {
    private final String title;
    private final int duration;
```

```

private final int passingScore;
private final boolean negativeMarking;
private final boolean questionShuffle;
private final boolean autoSubmit;
private final boolean calculatorAllowed;

// Private constructor: outside classes cannot directly create ExamConfiguration.
private ExamConfiguration(Builder builder) {
    this.title = builder.title;
    this.duration = builder.duration;
    this.passingScore = builder.passingScore;
    this.negativeMarking = builder.negativeMarking;
    this.questionShuffle = builder.questionShuffle;
    this.autoSubmit = builder.autoSubmit;
    this.calculatorAllowed = builder.calculatorAllowed;
}

public String getTitle() {
    return title;
}

public int getDuration() {
    return duration;
}

public int getPassingScore() {
    return passingScore;
}

public boolean isNegativeMarking() {
    return negativeMarking;
}

public boolean isQuestionShuffle() {
    return questionShuffle;
}

public boolean isAutoSubmit() {
    return autoSubmit;
}

public boolean isCalculatorAllowed() {
    return calculatorAllowed;
}

public void displayConfiguration() {
    System.out.println("Title: " + title);
    System.out.println("Duration: " + duration + " minutes");
    System.out.println("Passing Score: " + passingScore + " Marks");
    System.out.println("Configuration Profiles:");
    System.out.println((negativeMarking ? "✓" : "x") + " Negative Marking Enabled");
    System.out.println((questionShuffle ? "✓" : "x") + " Question Shuffle Activated");
    System.out.println((calculatorAllowed ? "✓" : "x") + " Embedded Calculator Allowed");
    System.out.println((autoSubmit ? "✓" : "x") + " Auto-Submit On Timeout");
}

public static class Builder {
    private String title;
    private int duration;
    private int passingScore;
    private boolean negativeMarking;
    private boolean questionShuffle;
    private boolean autoSubmit;
    private boolean calculatorAllowed;

    public Builder title(String title) {
        this.title = title;
        return this;
    }

    public Builder duration(int duration) {
        this.duration = duration;
        return this;
    }
}

```

```

public Builder passingScore(int passingScore) {
    this.passingScore = passingScore;
    return this;
}

public Builder enableNegativeMarking() {
    this.negativeMarking = true;
    return this;
}

public Builder disableNegativeMarking() {
    this.negativeMarking = false;
    return this;
}

public Builder enableQuestionShuffle() {
    this.questionShuffle = true;
    return this;
}

public Builder disableQuestionShuffle() {
    this.questionShuffle = false;
    return this;
}

public Builder enableAutoSubmit() {
    this.autoSubmit = true;
    return this;
}

public Builder disableAutoSubmit() {
    this.autoSubmit = false;
    return this;
}

public Builder enableCalculator() {
    this.calculatorAllowed = true;
    return this;
}

public Builder disableCalculator() {
    this.calculatorAllowed = false;
    return this;
}

public ExamConfiguration build() {
    validate();
    return new ExamConfiguration(this);
}

private void validate() {
    if (title == null || title.trim().isEmpty()) {
        throw new IllegalArgumentException("Exam title cannot be empty.");
    }

    if (duration <= 0) {
        throw new IllegalArgumentException("Exam duration must be greater than 0.");
    }

    if (passingScore < 0) {
        throw new IllegalArgumentException("Passing score cannot be negative.");
    }
}
}
}
}

```

How to Use It in Main.java

```

Java
public class Main {
    public static void main(String[] args) {

```

```

    ExamConfiguration config = new ExamConfiguration.Builder()
        .title("OOP Midterm Exam")
        .duration(60)
        .passingScore(50)
        .enableNegativeMarking()
        .enableQuestionShuffle()
        .enableAutoSubmit()
        .disableCalculator()
        .build();

    config.displayConfiguration();
}
}

```

Why Nested Builder Is Better

The constructor is private:

```

Java
private ExamConfiguration(Builder builder)

```

So nobody can do this:

```

Java
new ExamConfiguration("OOP Midterm", 60, 50, true, true, false, true);

```

That means the only way to create **ExamConfiguration** is through:

```

Java
new ExamConfiguration.Builder()

```

This protects your design.

Viva Explanation

Say this:

I used a nested static Builder inside **ExamConfiguration**. The builder temporarily stores the configuration values. Each builder method returns **this**, so method chaining becomes possible. Finally, **build()** validates the values and passes the builder to the private **ExamConfiguration** constructor. This makes the final configuration object immutable and prevents direct use of a long, confusing constructor.

That explanation is strong and examiner-friendly.

One small note: your assignment mentions **ExamBuilder**, but using **ExamConfiguration.Builder** is still a valid Builder Pattern implementation because the builder role is present and the construction is step-by-step.

DP Lab Assignment (Factory, Abs...

next milestone

Milestone 4 is where your design-review skill really matters. The danger here is accidentally destroying the Abstract Factory consistency we built in Milestone 2, so we'll treat this as a careful refactor, not just "add two classes."

Good. We move to **Milestone 4: Question Sourcing + Dependency Injection**.

This milestone connects with the assignment's Part D. The assignment says concrete question factories must accept an abstract **QuestionSource** through dependency injection, and the system should support sources like **Question Bank Mode** and **Faker Mode**.

DP Lab Assignment (Factory, Abs...

Milestone 4: Question Sourcing + Dependency Injection

What we already have

From Milestone 2, our factories create questions directly:

```
Java
public class MCQFactory implements QuestionFactory {

    @Override
    public Question createQuestion() {
        return new MCQQuestion(
            "Which design pattern creates families of related objects?",
            new String[]{"Factory Method", "Abstract Factory", "Builder", "Singleton"},
            "B",
            2,
            "Medium"
        );
    }
}
```

This works, but the factory is doing **two jobs**:

1. Creating the MCQ family
2. Deciding where the question data comes from

That is not ideal.

Main Idea of Milestone 4

We want to separate these two responsibilities:

QuestionFactory:
Responsible for creating a compatible question family.

QuestionSource:
Responsible for deciding where question data comes from.

So after this milestone, the factory should not care whether the question comes from:

Manual input
Question bank
Faker/generated content

The factory should only ask:

```
Java
source.getQuestion();
```

Before Milestone 4

```
Java
QuestionFactory factory = new MCQFactory();
Question question = factory.createQuestion();
```

The `MCQFactory` internally creates a hardcoded MCQ.

After Milestone 4

```
Java
QuestionSource source = new BankQuestionSource("MCQ");
QuestionFactory factory = new MCQFactory(source);
Question question = factory.createQuestion();
```

Now the client decides the source.

This is Dependency Injection:

```
MCQFactory does not create its own source.
The source is passed into MCQFactory from outside.
```

Important Design Warning

Do **not** do this inside `MCQFactory`:

```
Java
public class MCQFactory implements QuestionFactory {
    private QuestionSource source = new BankQuestionSource("MCQ");
}
```

Why is this bad?

Because the factory becomes locked to Bank mode.

Then this becomes difficult:

```
Java
new MCQFactory(new FakerQuestionSource("MCQ"));
```

So the correct design is:

```
Java
public class MCQFactory implements QuestionFactory {
    private final QuestionSource source;
```

```
public MCQFactory(QuestionSource source) {
    this.source = source;
}
}
```

Now the source can change without changing the factory.

The mental model

Think of it like this:

Abstract Factory answers:
Which compatible question family should I create?

QuestionSource answers:
Where should the question data come from?

So these are different responsibilities.

Example:

```
MCQFactory + BankQuestionSource
= MCQ question from question bank

MCQFactory + FakerQuestionSource
= MCQ question from generated fake content

EssayFactory + BankQuestionSource
= Essay question from question bank

EssayFactory + FakerQuestionSource
= Essay question from generated fake content
```

This is powerful because we can change the source without rewriting the question family classes.

Read / Understand First

For this milestone, focus on these ideas:

```
Dependency Injection
Inversion of Control
Strategy-like source switching
Constructor injection
```

The core sentence:

Instead of a class creating its own dependency, the dependency is passed into the class from outside.

Milestone 4 Design Target

We will add:

```
QuestionSource
├── BankQuestionSource
```

└─ FakerQuestionSource

Then we will refactor:

```
MCQFactory
TrueFalseFactory
EssayFactory
ProgrammingFactory
```

so each accepts:

```
Java
QuestionSource source
```

through its constructor.

Milestone 4 MCQs: Design-Trap Version

Answer with **letter + one sentence reasoning**.

Q1.

Which version shows proper dependency injection?

A)

```
Java
public class MCQFactory {
    private QuestionSource source = new BankQuestionSource("MCQ");
}
```

B)

```
Java
public class MCQFactory {
    private final QuestionSource source;

    public MCQFactory(QuestionSource source) {
        this.source = source;
    }
}
```

C)

```
Java
public class MCQFactory {
    public Question createQuestion() {
        return new MCQQuestion();
    }
}
```

D)

```
Java
public class MCQFactory {
    public void printSource() {
        System.out.println("Bank");
    }
}
```

Q2.

Why is option A from Q1 weaker?

- A) It makes the factory permanently dependent on one concrete source
- B) It makes Java unable to compile
- C) It removes inheritance
- D) It creates too many interfaces

Q3.

What should **QuestionSource** be responsible for?

- A) Rendering the question
- B) Evaluating the answer
- C) Supplying question data or question objects from a chosen origin
- D) Creating the final exam configuration

Q4.

What should **MCQFactory** still be responsible for after adding **QuestionSource**?

- A) Choosing whether the source is Bank or Faker
- B) Creating compatible MCQ family objects: question, renderer, evaluator
- C) Printing the final dashboard
- D) Building exam configuration

Q5.

Which design keeps source switching flexible?

- A)

```
Java
QuestionFactory factory = new MCQFactory(new BankQuestionSource("MCQ"));
```

and later:

```
Java
```

```
QuestionFactory factory = new MCQFactory(new FakerQuestionSource("MCQ"));
```

B)

Java

MCQFactory always creates BankQuestionSource internally.

C)

Java

MCQRenderer creates questions from the bank.

D)

Java

ExamConfiguration decides where questions come from.

Q6.

What is the main danger if `BankQuestionSource("MCQ")` accidentally returns an `EssayQuestion`?

- A) It breaks family consistency
- B) It makes the builder immutable
- C) It improves abstraction
- D) It removes the need for evaluator

Q7.

Which statement is most accurate?

- A) Dependency Injection replaces Abstract Factory
- B) Dependency Injection supports Abstract Factory by giving it flexible input sources
- C) Abstract Factory and Dependency Injection are the same pattern
- D) Dependency Injection is only useful for databases

Q8.

If the assignment later asks for `ApiQuestionSource`, what should we ideally do?

- A) Modify all existing factories
- B) Create `ApiQuestionSource implements QuestionSource` and inject it where needed
- C) Delete `QuestionFactory`
- D) Move everything into `Main.java`

Q9.

Which one violates separation of responsibility?

- A) **QuestionSource** supplies question content
- B) **MCQRenderer** displays MCQ format
- C) **MCQEvaluator** checks MCQ answer
- D) **MCQFactory** reads user input, fetches database data, renders UI, and evaluates answers

Q10.

Why is constructor injection better here than creating the source inside the factory?

- A) The client can choose the source at runtime
- B) It makes code impossible to test
- C) It forces every source to be static
- D) It removes all object creation from Java

Send your answers like this:

Q1. B – Because the source is passed from outside, not hardcoded inside the factory.
Q2. ...

After I mark them, we will refactor the Milestone 2 code into proper **QuestionSource + dependency injection** style.

Q1. B — Because the QuestionSource is passed from outside, so the factory is not locked to one source.

Q2. A — Because MCQFactory becomes permanently dependent on BankQuestionSource, which makes source switching hard.

Q3. C — Because QuestionSource should supply question data or question objects from a selected origin like Bank, Faker, or Manual.

Q4. B — Because MCQFactory should still create the compatible MCQ family: question, renderer, and evaluator.

Q5. A — Because we can inject BankQuestionSource now and replace it with FakerQuestionSource later without changing MCQFactory.

Q6. A — Because returning an EssayQuestion from an MCQ source/factory breaks the matching family idea.

Q7. B — Because Dependency Injection does not replace Abstract Factory; it supports it by giving flexible input sources.

Q8. B — Because we only need to create ApiQuestionSource implements QuestionSource and

inject it where needed.

Q9. D — Because one class doing input, database fetching, rendering, and evaluation violates separation of responsibility.

Q10. A — Because constructor injection lets the client choose the source at runtime instead of hardcoding it inside the factory.

Show moreShow less

Your MCQ reasoning is strong: 10/10. The only subtle design issue now is deciding whether **QuestionSource** should return a full **Question** object or only raw question data; the assignment skeleton says **Question** `getQuestion()`, so we'll follow that while adding validation to protect family consistency.

Excellent. Your Milestone 4 MCQ score: **10/10** 

Your reasoning is strong, especially Q7 and Q10. You correctly understood that **Dependency Injection does not replace Abstract Factory**. It supports it by making the source of question data flexible.

Now we refactor the Milestone 2 code.

Your assignment says concrete question factories must accept an abstract **QuestionSource** through dependency injection, and it gives this structure:

```
Java
interface QuestionSource {
    Question getQuestion();
}
```

The goal is to switch between sources such as **Question Bank Mode** and **Faker Mode** without rewriting the factories.

DP Lab Assignment (Factory, Abs...

Milestone 4 Code: Question Sourcing + Dependency Injection

New structure

Add these new files:

```
QuestionSource.java
BankQuestionSource.java
FakerQuestionSource.java
QuestionBank.java
QuestionFaker.java
```

Then update these old files:

```
MCQFactory.java
TrueFalseFactory.java
EssayFactory.java
ProgrammingFactory.java
Main.java
```

1. QuestionSource.java

```
Java
public interface QuestionSource {
    Question getQuestion();

    String getSourceName();
}
```

The assignment only requires:

```
Java
Question getQuestion();
```

I added:

```
Java
String getSourceName();
```

because the final dashboard must print:

```
Question Sourcing Strategy: Question Bank Mode
```

So this will help us later in Milestone 5.

2. BankQuestionSource.java

```
Java
public class BankQuestionSource implements QuestionSource {
    private final String questionType;

    public BankQuestionSource(String questionType) {
        this.questionType = questionType;
    }

    @Override
    public Question getQuestion() {
        return QuestionBank.getRandomQuestion(questionType);
    }

    @Override
    public String getSourceName() {
        return "Question Bank Mode";
    }
}
```

This class does not create renderers or evaluators.

Its only job is:

```
Supply a question from the question bank.
```

3. FakerQuestionSource.java

Java

```
public class FakerQuestionSource implements QuestionSource {
    private final String questionType;

    public FakerQuestionSource(String questionType) {
        this.questionType = questionType;
    }

    @Override
    public Question getQuestion() {
        return QuestionFaker.generateQuestion(questionType);
    }

    @Override
    public String getSourceName() {
        return "Faker Mode";
    }
}
```

This class generates fake/synthetic question content.

4. QuestionBank.java

Java

```
import java.util.Arrays;
import java.util.List;
import java.util.Random;

public class QuestionBank {
    private static final Random random = new Random();

    public static Question getRandomQuestion(String questionType) {
        List<Question> questions = getQuestionsByType(questionType);

        int index = random.nextInt(questions.size());
        return questions.get(index);
    }

    private static List<Question> getQuestionsByType(String questionType) {
        if (questionType.equalsIgnoreCase("MCQ")) {
            return Arrays.asList(
                new MCQQuestion(
                    "Which design pattern creates families of related objects?",
                    new String[]{"Factory Method", "Abstract Factory", "Builder",
"Singleton"},
                    "B",
                    2,
                    "Medium"
                ),
                new MCQQuestion(
                    "Which pattern is best for building complex objects step by step?",
                    new String[]{"Adapter", "Builder", "Observer", "Proxy"},
                    "B",
                    2,
                    "Easy"
                )
            );
        }

        if (questionType.equalsIgnoreCase("TRUE_FALSE")) {
            return Arrays.asList(
                new TrueFalseQuestion(
                    "Factory Method and Abstract Factory are both creational patterns.",
                    true,
                    1,
                    "Easy"
                ),
                new TrueFalseQuestion(

```

```

        "Builder is mainly used to create families of related objects.",
        false,
        1,
        "Medium"
    );
}

if (questionType.equalsIgnoreCase("ESSAY")) {
    return Arrays.asList(
        new EssayQuestion(
            "Explain why Abstract Factory is useful in an online exam system.",
            new String[]{"family", "renderer", "evaluator"},
            10,
            "Hard"
        ),
        new EssayQuestion(
            "Explain the benefit of dependency injection in question sourcing.",
            new String[]{"source", "injection", "flexible"},
            10,
            "Medium"
        )
    );
}

if (questionType.equalsIgnoreCase("PROGRAMMING")) {
    return Arrays.asList(
        new ProgrammingQuestion(
            "Write a Java method that returns the larger of two numbers.",
            "Math.max",
            20,
            "Hard"
        ),
        new ProgrammingQuestion(
            "Write a Java method that checks whether a number is even.",
            "%",
            20,
            "Medium"
        )
    );
}

throw new IllegalArgumentException("Unknown question type: " + questionType);
}
}

```

5. QuestionFaker.java

```

Java
import java.util.Random;

public class QuestionFaker {
    private static final Random random = new Random();

    public static Question generateQuestion(String questionType) {
        int randomNumber = random.nextInt(1000);

        if (questionType.equalsIgnoreCase("MCQ")) {
            return new MCQQuestion(
                "Generated MCQ #" + randomNumber + ": Which concept hides object creation
logic?",
                new String[]{"Inheritance", "Factory Method", "Looping", "Exception
Handling"},
                "B",
                2,
                "Medium"
            );
        }

        if (questionType.equalsIgnoreCase("TRUE_FALSE")) {
            return new TrueFalseQuestion(

```

```

tight coupling.",
    true,
    1,
    "Easy"
);
}

if (questionType.equalsIgnoreCase("ESSAY")) {
    return new EssayQuestion(
        "Generated Essay #" + randomNumber + ": Discuss how design patterns improve
maintainability.",
        new String[]{"design", "maintainability", "flexible"},
        10,
        "Hard"
    );
}

if (questionType.equalsIgnoreCase("PROGRAMMING")) {
    return new ProgrammingQuestion(
        "Generated Programming #" + randomNumber + ": Write a method that returns the
smaller of two numbers.",
        "Math.min",
        20,
        "Hard"
    );
}

throw new IllegalArgumentException("Unknown question type: " + questionType);
}
}

```

Now Refactor the Factories

Previously, factories created hardcoded questions.

Now each factory receives a `QuestionSource`.

6. Updated MCQFactory.java

```

Java
public class MCQFactory implements QuestionFactory {
    private final QuestionSource source;

    public MCQFactory(QuestionSource source) {
        this.source = source;
    }

    @Override
    public Question createQuestion() {
        Question question = source.getQuestion();

        if (!(question instanceof MCQQuestion)) {
            throw new IllegalStateException("MCQFactory expected MCQQuestion but received " +
question.getType());
        }

        return question;
    }

    @Override
    public QuestionRenderer createRenderer() {
        return new MCQRenderer();
    }

    @Override
    public QuestionEvaluator createEvaluator() {

```

```
        return new MCQEvaluator();
    }
}
```

The important part is:

```
Java
public MCQFactory(QuestionSource source) {
    this.source = source;
}
```

This is constructor injection.

7. Updated TrueFalseFactory.java

```
Java
public class TrueFalseFactory implements QuestionFactory {
    private final QuestionSource source;

    public TrueFalseFactory(QuestionSource source) {
        this.source = source;
    }

    @Override
    public Question createQuestion() {
        Question question = source.getQuestion();

        if (!(question instanceof TrueFalseQuestion)) {
            throw new IllegalStateException("TrueFalseFactory expected TrueFalseQuestion but
received " + question.getType());
        }

        return question;
    }

    @Override
    public QuestionRenderer createRenderer() {
        return new TrueFalseRenderer();
    }

    @Override
    public QuestionEvaluator createEvaluator() {
        return new TrueFalseEvaluator();
    }
}
```

8. Updated EssayFactory.java

```
Java
public class EssayFactory implements QuestionFactory {
    private final QuestionSource source;

    public EssayFactory(QuestionSource source) {
        this.source = source;
    }

    @Override
    public Question createQuestion() {
        Question question = source.getQuestion();

        if (!(question instanceof EssayQuestion)) {
            throw new IllegalStateException("EssayFactory expected EssayQuestion but received " +
question.getType());
        }
    }
}
```

```

    }

    return question;
}

@Override
public QuestionRenderer createRenderer() {
    return new EssayRenderer();
}

@Override
public QuestionEvaluator createEvaluator() {
    return new EssayEvaluator();
}
}

```

9. Updated ProgrammingFactory.java

Java

```

public class ProgrammingFactory implements QuestionFactory {
    private final QuestionSource source;

    public ProgrammingFactory(QuestionSource source) {
        this.source = source;
    }

    @Override
    public Question createQuestion() {
        Question question = source.getQuestion();

        if (!(question instanceof ProgrammingQuestion)) {
            throw new IllegalStateException("ProgrammingFactory expected ProgrammingQuestion but
received " + question.getType());
        }

        return question;
    }

    @Override
    public QuestionRenderer createRenderer() {
        return new ProgrammingRenderer();
    }

    @Override
    public QuestionEvaluator createEvaluator() {
        return new ProgrammingEvaluator();
    }
}

```

10. Temporary Main.java for Testing Milestone 4

Java

```

public class Main {
    public static void main(String[] args) {

        QuestionSource mcqBankSource = new BankQuestionSource("MCQ");
        QuestionSource essayFakerSource = new FakerQuestionSource("ESSAY");

        QuestionFactory mcqFactory = new MCQFactory(mcqBankSource);
        QuestionFactory essayFactory = new EssayFactory(essayFakerSource);

        testQuestionFamily(mcqFactory, "B", mcqBankSource);
        testQuestionFamily(essayFactory, "Design patterns make systems flexible and
maintainable.", essayFakerSource);
    }

    private static void testQuestionFamily(
        QuestionFactory factory,

```

```

        String sampleAnswer,
        QuestionSource source
    ) {
        Question question = factory.createQuestion();
        QuestionRenderer renderer = factory.createRenderer();
        QuestionEvaluator evaluator = factory.createEvaluator();

        System.out.println("=====");
        System.out.println("Source: " + source.getSourceName());
        renderer.render(question);

        int score = evaluator.evaluate(question, sampleAnswer);

        System.out.println("Type: " + question.getType());
        System.out.println("Points: " + question.getPoints());
        System.out.println("Difficulty: " + question.getDifficulty());
        System.out.println("Sample Score: " + score);
        System.out.println("=====");
        System.out.println();
    }
}

```

Expected Output

Your exact question may vary because the bank and faker use random generation, but the structure should look like:

```

=====
Source: Question Bank Mode
[MCQ Question]
Which design pattern creates families of related objects?
A) Factory Method
B) Abstract Factory
C) Builder
D) Singleton
Type: MCQ
Points: 2
Difficulty: Medium
Sample Score: 2
=====

=====
Source: Faker Mode
[Essay Question]
Generated Essay #421: Discuss how design patterns improve maintainability.
Write a descriptive answer.
Type: Essay
Points: 10
Difficulty: Hard
Sample Score: 5
=====

```

Why this is Dependency Injection

This is not dependency injection:

```

Java
public class MCQFactory implements QuestionFactory {
    private QuestionSource source = new BankQuestionSource("MCQ");
}

```

Because `MCQFactory` chooses its own dependency.

This is dependency injection:

```
Java
public class MCQFactory implements QuestionFactory {
    private final QuestionSource source;

    public MCQFactory(QuestionSource source) {
        this.source = source;
    }
}
```

Because the source is passed from outside.

So the client can choose:

```
Java
new MCQFactory(new BankQuestionSource("MCQ"));
```

or:

```
Java
new MCQFactory(new FakerQuestionSource("MCQ"));
```

without changing `MCQFactory`.

Design Explanation for Viva

Say this:

In Milestone 4, I added `QuestionSource` to decouple question creation from the origin of question data. `BankQuestionSource` supplies questions from a predefined question bank, while `FakerQuestionSource` generates simulated questions at runtime. Each concrete question factory receives a `QuestionSource` through constructor injection. This means the factory still creates the correct question family, but it no longer knows or cares whether the question came from a bank, faker, or another future source.

Stronger version:

Dependency Injection supports the Abstract Factory here. Abstract Factory keeps the question, renderer, and evaluator compatible. Dependency Injection keeps the source of the question flexible. So these two ideas solve different problems but work together.

Design Review Warning

There is one subtle weakness in the assignment's skeleton:

```
Java
interface QuestionSource {
    Question getQuestion();
}
```

Because `QuestionSource` returns a full `Question`, a badly written source could return the wrong question type.

Example:

```
Java
new MCQFactory(new BankQuestionSource("ESSAY"));
```

That would be wrong.

That is why we added this validation:

```
Java
if (!(question instanceof MCQQuestion)) {
    throw new IllegalStateException("MCQFactory expected MCQQuestion but received " +
question.getType());
}
```

This protects the family consistency.

Milestone 4 Complete

You now understand:

```
Abstract Factory = creates compatible families
Dependency Injection = injects flexible source from outside
QuestionSource = decides where question data comes from
Concrete Factory = still controls family compatibility
```

Next milestone is **Milestone 5: Integration, Orchestration & Deliverables.**

In Milestone 5 we will wire together:

```
ExamFactory
QuestionFactory
QuestionSource
ExamConfiguration.Builder
Main.java dashboard
UML explanation
Submission checklist
```

next milestone

Great. **Milestone 5 starts now: Integration, Orchestration & Deliverables.**

This milestone is not about learning a new pattern. It is about proving that all previous patterns work together inside one clean system.

Your assignment says **Main.java** must: choose an **ExamFactory**, choose a **QuestionSource**, initialize question factories with that source, build configuration through builder chaining, and print a dashboard-style final report. It also requires a UML diagram, GitHub/source link, AI-chat record, and explanation ability.

DP Lab Assignment (Factory, Abs...

Milestone 5: Integration Mindset

What we have already built

Factory Method:
ExamFactory → creates **PracticeQuiz** / **MidtermExam** / **FinalExam**

Abstract Factory:
QuestionFactory → creates **Question** + **Renderer** + **Evaluator** family

Builder:
ExamConfiguration.Builder → creates immutable exam configuration

Dependency Injection:
QuestionSource → injected into question factories

Now **Main.java** will connect them.

Key Concept: Main.java Is the Orchestrator

Main.java should not contain heavy business logic.

It should only coordinate:

1. Select exam type
2. Select question source
3. Create question factories
4. Build exam configuration
5. Collect generated questions
6. Print dashboard

Think of **Main.java** like a project manager.

It should not become:

Question creator
Question evaluator
Renderer
Configuration object
Question bank
Faker

It should just wire the system together.

Final Architecture Flow

```

Main.java
  ↓
ExamFactory
  ↓
Exam

Main.java
  ↓
QuestionSource
  ↓
QuestionFactory
  ↓
Question + Renderer + Evaluator

Main.java
  ↓

```

```
ExamConfiguration.Builder
↓
ExamConfiguration
Main.java
↓
Dashboard Output
```

Most Important Design Danger

This is bad integration:

```
Java
Question question = new MCQQuestion(...);
QuestionRenderer renderer = new EssayRenderer();
QuestionEvaluator evaluator = new ProgrammingEvaluator();
```

Because it destroys the Abstract Factory family consistency.

This is better:

```
Java
QuestionFactory factory = new MCQFactory(new BankQuestionSource("MCQ"));

Question question = factory.createQuestion();
QuestionRenderer renderer = factory.createRenderer();
QuestionEvaluator evaluator = factory.createEvaluator();
```

Now the family stays together.

Milestone 5 MCQs: Final Design Review

Answer with **letter + one sentence reasoning**.

Q1.

What should be the main responsibility of `Main.java` in the final project?

- A) Implement all question evaluation logic
- B) Orchestrate object creation and print the final dashboard
- C) Store all question bank data
- D) Replace all factories

Q2.

Which line best shows Factory Method integration?

A)

```
Java
Exam exam = new MidtermExam();
```

B)

Java

```
ExamFactory examFactory = new MidtermExamFactory();
Exam exam = examFactory.createExam();
```

C)

Java

```
QuestionSource source = new BankQuestionSource("MCQ");
```

D)

Java

```
ExamConfiguration config = null;
```

Q3.

Which line best shows Builder integration?

A)

Java

```
ExamConfiguration config = new ExamConfiguration("Midterm", 60, 50, true, true, true, false);
```

B)

Java

```
ExamConfiguration config = new ExamConfiguration.Builder()
    .title("OOP Midterm Exam")
    .duration(60)
    .passingScore(50)
    .enableNegativeMarking()
    .enableQuestionShuffle()
    .enableAutoSubmit()
    .disableCalculator()
    .build();
```

C)

Java

```
Exam exam = factory.createExam();
```

D)

Java

```
QuestionRenderer renderer = factory.createRenderer();
```

Q4.

Which line best shows Dependency Injection?

A)

```
Java
public class MCQFactory {
    private QuestionSource source = new BankQuestionSource("MCQ");
}
```

B)

```
Java
QuestionFactory mcqFactory = new MCQFactory(new BankQuestionSource("MCQ"));
```

C)

```
Java
Question question = new MCQQuestion();
```

D)

```
Java
System.out.println("MCQ");
```

Q5.

In the final dashboard, where should question type, points, and difficulty come from?

- A) Hardcoded strings in `Main.java`
- B) The `Question` objects generated by the factories
- C) The `ExamFactory` only
- D) The `ExamConfiguration` only

Q6.

Which final integration design is better?

A)

```
Java
QuestionFactory mcqFactory = new MCQFactory(source);
Question mcq = mcqFactory.createQuestion();
```

B)

```
Java
Question mcq = new MCQQuestion(...);
```

C)

```
Java
```

```
Question mcq = new EssayQuestion(...);
```

D)

```
Java
String mcq = "MCQ";
```

Q7.

Why should we store generated questions in a list before printing the dashboard?

- A) So we can treat all question types uniformly through the **Question** interface
- B) Because Java cannot print objects directly
- C) Because lists are required by Factory Method
- D) Because Builder needs a list

Q8.

Which one is a violation of separation of responsibility?

- A) **ExamFactory** creates an exam
- B) **QuestionSource** supplies question data
- C) **ExamConfiguration.Builder** builds configuration
- D) **Main.java** manually evaluates every answer using large **if-else** blocks

Q9.

Suppose the final output says:

```
Question Sourcing Strategy: Question Bank Mode
```

Which object should provide this text most cleanly?

- A) **QuestionSource** through **getSourceName()**
- B) **MCQRenderer**
- C) **ExamFactory**
- D) **ExamConfiguration.Builder**

Q10.

What is the best explanation of the whole project architecture?

- A) One big class creates and handles everything
- B) Each pattern solves a separate creation problem, and **Main.java** combines them

- C) Builder replaces Abstract Factory
- D) Dependency Injection removes the need for interfaces

Q11.

If we add **CertificationExam**, which part should mainly change?

- A) Add **CertificationExam** and **CertificationExamFactory**
- B) Rewrite all question factories
- C) Rewrite **ExamConfiguration**
- D) Delete **QuestionSource**

Q12.

If we add **FillBlank** question type, which part should mainly change?

- A) Add **FillBlankQuestion**, **FillBlankRenderer**, **FillBlankEvaluator**, and **FillBlankFactory**
- B) Add only **FillBlankQuestion**
- C) Rewrite **MidtermExamFactory**
- D) Remove Builder

Send your answers like this:

Q1. B – Because **Main.java** should coordinate the system, not contain all business logic.
Q2. ...

After you answer, we'll write the final integrated **Main.java**, dashboard printer logic, and then prepare the UML/explanation checklist.

Q1. B

Q2. B

Q3. B

Q4. B

Q5. B

Q6. A

Q7. A

Q8. D .

Q9. A

Q10. B

Q11. A

Q12. A
 Show moreShow less

Perfect. Milestone 5 MCQ score: **12/12** 

You selected all correct answers. This means you understand the final design idea: **Main.java should orchestrate, not do the work itself.**

Your assignment requires the final client workflow to instantiate an **ExamFactory**, choose a **QuestionSource**, initialize question factories with that source strategy, build configuration through Builder chaining, and print a dashboard-style report.

DP Lab Assignment (Factory, Abs...

Final Integrated Main.java

Use this version for the final project integration.

```
Java
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {

        // 1. Factory Method: choose exam type
        ExamFactory examFactory = new MidtermExamFactory();
```

```

Exam exam = examFactory.createExam();

// 2. Question Source Strategy: choose sourcing mode
// We are using Bank mode for all question types in this final demo.
QuestionSource mcqSource = new BankQuestionSource("MCQ");
QuestionSource essaySource = new BankQuestionSource("ESSAY");
QuestionSource programmingSource = new BankQuestionSource("PROGRAMMING");
QuestionSource trueFalseSource = new BankQuestionSource("TRUE_FALSE");

String sourceStrategyName = mcqSource.getSourceName();

// 3. Abstract Factory + Dependency Injection
QuestionFactory mcqFactoryOne = new MCQFactory(mcqSource);
QuestionFactory mcqFactoryTwo = new MCQFactory(new BankQuestionSource("MCQ"));
QuestionFactory essayFactory = new EssayFactory(essaySource);
QuestionFactory programmingFactory = new ProgrammingFactory(programmingSource);
QuestionFactory trueFalseFactory = new TrueFalseFactory(trueFalseSource);

// 4. Builder Pattern: build immutable exam configuration
ExamConfiguration config = new ExamConfiguration.Builder()
    .title("OOP Midterm Exam")
    .duration(60)
    .passingScore(50)
    .enableNegativeMarking()
    .enableQuestionShuffle()
    .enableAutoSubmit()
    .disableCalculator()
    .build();

// 5. Generate questions
List<Question> questions = new ArrayList<>();

questions.add(mcqFactoryOne.createQuestion());
questions.add(mcqFactoryTwo.createQuestion());
questions.add(essayFactory.createQuestion());
questions.add(programmingFactory.createQuestion());
questions.add(trueFalseFactory.createQuestion());

// 6. Print final dashboard
printDashboard(exam, config, sourceStrategyName, questions);
}

private static void printDashboard(
    Exam exam,
    ExamConfiguration config,
    String sourceStrategyName,
    List<Question> questions
) {

System.out.println("=====");
    System.out.println("EXAM CREATED SUCCESSFULLY");

System.out.println("=====");

    System.out.println("Type: " + exam.getType());
    System.out.println("Title: " + config.getTitle());
    System.out.println("Duration: " + config.getDuration() + " minutes");
    System.out.println("Passing Score: " + config.getPassingScore() + " Marks");

    System.out.println("Configuration Profiles:");
    System.out.println((config.isNegativeMarking() ? "✓" : "x") + " Negative Marking
Enabled");
    System.out.println((config.isQuestionShuffle() ? "✓" : "x") + " Question Shuffle
Activated");
    System.out.println((config.isCalculatorAllowed() ? "✓" : "x") + " Embedded Calculator
Allowed");
    System.out.println((config.isAutoSubmit() ? "✓" : "x") + " Auto-Submit On Timeout");

    System.out.println("Question Sourcing Strategy: " + sourceStrategyName);

    System.out.println("Compiled Structural Components:");

    for (Question question : questions) {
        System.out.println(
            "- [Type: " + question.getType() + "] "

```

```

        + "Points: " + question.getPoints()
        + " Difficulty: " + question.getDifficulty()
    );
}

System.out.println("=====");
}
}

```

Expected Dashboard Output

Your exact MCQ order may vary because the bank uses random selection, but the final output should look like this:

```

=====
EXAM CREATED SUCCESSFULLY
=====
Type: Midterm Exam
Title: OOP Midterm Exam
Duration: 60 minutes
Passing Score: 50 Marks
Configuration Profiles:
✓ Negative Marking Enabled
✓ Question Shuffle Activated
✗ Embedded Calculator Allowed
✓ Auto-Submit On Timeout
Question Sourcing Strategy: Question Bank Mode
Compiled Structural Components:
- [Type: MCQ] Points: 2 Difficulty: Medium
- [Type: MCQ] Points: 2 Difficulty: Easy
- [Type: Essay] Points: 10 Difficulty: Hard
- [Type: Programming] Points: 20 Difficulty: Hard
- [Type: True/False] Points: 1 Difficulty: Easy
=====

```

This matches the assignment's required dashboard structure.

DP Lab Assignment (Factory, Abs...

Why This Final Integration Is Correct

1. Factory Method is used here

```

Java
ExamFactory examFactory = new MidtermExamFactory();
Exam exam = examFactory.createExam();

```

This means `Main.java` does not directly create:

```

Java
new MidtermExam();

```

Instead, it asks the factory to create the exam.

2. Abstract Factory is used here

Java

```
QuestionFactory essayFactory = new EssayFactory(essaySource);
Question question = essayFactory.createQuestion();
```

Each question factory creates a compatible family:

```
EssayQuestion
EssayRenderer
EssayEvaluator
```

Even though the dashboard only prints question details, the renderer and evaluator are still part of the design.

3. Builder is used here

Java

```
ExamConfiguration config = new ExamConfiguration.Builder()
    .title("OOP Midterm Exam")
    .duration(60)
    .passingScore(50)
    .enableNegativeMarking()
    .enableQuestionShuffle()
    .enableAutoSubmit()
    .disableCalculator()
    .build();
```

This avoids the confusing constructor:

Java

```
new ExamConfiguration("OOP Midterm", 60, 50, true, true, true, false);
```

The nested builder also protects the immutable configuration object.

4. Dependency Injection is used here

Java

```
QuestionFactory mcqFactoryOne = new MCQFactory(mcqSource);
```

The factory does not create its own source. The source is passed from outside.

That means this can change easily:

Java

```
new MCQFactory(new BankQuestionSource("MCQ"));
```

to this:

Java

```
new MCQFactory(new FakerQuestionSource("MCQ"));
```

without rewriting **MCQFactory**.

Final Viva Explanation

Use this:

My final system combines three creational design patterns. I used Factory Method for exam provisioning, where **ExamFactory** creates different exam types like Practice Quiz, Midterm, and Final. I used Abstract Factory for the question subsystem because each question type has a related family: question entity, renderer, and evaluator. I used Builder for **ExamConfiguration** because the configuration has many optional settings, and the builder makes construction readable and safe. Finally, I used dependency injection through **QuestionSource** so the factories can receive questions from Bank mode or Faker mode without knowing where the data comes from.

Stronger version:

Main.java only orchestrates the system. It selects the exam factory, selects the question source, injects the source into question factories, builds the exam configuration, collects generated questions, and prints the final dashboard. The creation responsibilities remain separated across the correct classes.

UML Class Diagram Checklist

Your UML should include these groups.

Factory Method group

```

Exam
├── PracticeQuiz
├── MidtermExam
└── FinalExam

ExamFactory
├── PracticeQuizFactory
├── MidtermExamFactory
└── FinalExamFactory
  
```

Relationship:

```

PracticeQuiz implements Exam
MidtermExam implements Exam
FinalExam implements Exam

PracticeQuizFactory extends ExamFactory
MidtermExamFactory extends ExamFactory
FinalExamFactory extends ExamFactory

ExamFactory creates Exam
  
```

Abstract Factory group

```

Question
├── MCQQuestion
├── TrueFalseQuestion
├── EssayQuestion
└── ProgrammingQuestion

QuestionRenderer
├── MCQRenderer
├── TrueFalseRenderer
├── EssayRenderer
└── ProgrammingRenderer

QuestionEvaluator
├── MCQEvaluator
├── TrueFalseEvaluator
├── EssayEvaluator
└── ProgrammingEvaluator

QuestionFactory
├── MCQFactory
├── TrueFalseFactory
├── EssayFactory
└── ProgrammingFactory

```

Relationship:

```

QuestionFactory creates Question
QuestionFactory creates QuestionRenderer
QuestionFactory creates QuestionEvaluator

```

Builder group

```

ExamConfiguration
├── static Builder

```

Relationship:

```

Builder builds ExamConfiguration
ExamConfiguration has private final fields

```

Question Source group

```

QuestionSource
├── BankQuestionSource
└── FakerQuestionSource

QuestionBank
QuestionFaker

```

Relationship:

```

BankQuestionSource uses QuestionBank
FakerQuestionSource uses QuestionFaker
QuestionFactory receives QuestionSource

```

Submission Checklist

Before submitting, prepare:

1. Full Java source code
2. GitHub public repository link
3. UML class diagram
4. Screenshot of final console output
5. PDF named after your BSSE roll
6. AI chat record
7. Short explanation of each pattern

Your project is now conceptually complete.